Artificial Intelligence and Robotics

A Complete Course in Plain English

ARTIFICIAL INTELLIGENCE AND ROBOTICS

Volume One

Introduction to Artificial Intelligence

TBUGE MECHATRONIX

Artificial Intelligence and Robotics, Volume Two: Robotics Fundamentals

First edition. TBUGE Mechatronix.

This volume assumes the vocabulary of Volume One: agents, sensors, actuators, policies and performance measures. It assumes nothing else. No electronics background is needed, no workshop is required, and every calculation can be done with a pencil and a calculator.

The series runs to forty-eight volumes, one for each course in the programme. Volume One asked what intelligence means in a machine. This one builds the machine that could have it.

Course codes have been left out on purpose. A subject should be named, not numbered.

Contents

Preface **6**

How This Volume Is Arranged **7**

Part One · The Machine and Its Body **9**

The Machine That Acts 10

Bodies: Links, Frames and Stiffness 15

Joints, Degrees of Freedom and Workspace 20

Part Two · Muscle: Motors, Gears and Power **25**

Electric Motors 26

Gearboxes, Belts and Screws 30

Power: Batteries, Wires and Heat 35

Electronics and Motor Drivers 40

Part Three · Sense: Instruments and Signals **45**

Sensors: Turning the World into Numbers 46

Signals, Noise and Sampling 50

Part Four · Behaviour: From Commands to Conduct **54**

Open Loop: Doing What You Are Told 55

Closing the Loop: Simple Behaviours That Survive 60

When Robots Hurt People 65

Part Five · Making and Using **70**

Building Your First Robot 71

Robots at Work and at Home 76

Glossary **81**

Sources and Further Reading **83**

Preface

Software can be wrong and then be fixed. A robot can be wrong once, at 2 m/s, in a room with a person in it, and the mistake is permanent. That asymmetry is what makes this subject different from programming, and it is the reason this volume exists before any of the clever ones.

A robot is a machine that acts. It has a body, which has mass and therefore momentum. It has joints, which bend and wear. It has muscles, which are motors that draw current, heat up and stall. It has senses, which are imperfect instruments that drift in the cold and pick up the hum of a nearby motor. It has a power supply that runs down. And it has a controller that must decide what to do with all of that in a few milliseconds, because the world does not pause while the software thinks.

None of this is background. It is the material. A vision system that finds a cup to a millimetre is useless if the arm cannot hold the cup steady. A planner that computes an elegant path is useless if the wheels slip on a polished floor. Half of the failures in robotics are failures of the body, and half of the rest are failures of the seam between body and code.

This volume is therefore about hardware, but it is written for people who intend to program. It does not assume you have a workshop, a 3D printer or a soldering iron. It assumes you have a pencil and the willingness to compute: torque on an arm, current through a motor, counts from an encoder, the energy in a battery, the time available in a control loop. Those five calculations, done honestly, will let you read a robot's specification sheet and know what it means, and let you design something that will actually work rather than something that looks like it should.

What you will be able to do

By the end of the volume you will be able to look at a robot — a factory arm, a vacuum cleaner, a drone, a delivery cart — and answer the questions an engineer asks first. How strong is it, and at what reach? How fast, and at what accuracy? What does it sense, and how good are those senses? What happens when it fails, and who is protected when it does? And what would it cost to build, in money and in months?

You will also be able to build a first robot of your own, or at least to specify one precisely enough that somebody else could build it: a list of parts, a wiring plan, a control loop, a test plan and a safety case.

A note on numbers

Every calculation in this volume uses ordinary units and is worked in full. Torque is in newton-metres, current in amperes, energy in joules and watt-hours, angles in degrees and radians. When a figure depends on an assumption, the assumption is stated. When a figure is a rough rule of thumb rather than a law of physics, it says so.

The arithmetic is deliberately small. You will not need calculus. You will need to multiply, divide, and keep track of units, which is 90% of engineering and 100% of the difference between a design that works and one that quietly destroys itself on the third day.

How This Volume Is Arranged

The volume follows the order in which a robot is actually built: the body first, then the muscles, then the nerves, then the behaviour, and finally the workshop and the world.

Part One asks what a robot is, and what a body has to do. It covers structure, stiffness, joints, degrees of freedom and workspace. By the end you will be able to count the freedom of a mechanism and predict what it cannot reach.

Part Two is about power: motors, gears, batteries, wiring and heat. These chapters contain the arithmetic that decides whether a design is possible, and they are the ones to read with a pencil.

Part Three is about nerve: drivers, electronics, sensors and signal quality. It is the part most often skipped and most often responsible for a machine that works on the bench and not on the floor.

Part Four is about behaviour. It starts with doing exactly what you are told, which is harder than it sounds, and moves to simple feedback that survives small disturbances. It ends with the chapter on safety, which is not an appendix to the subject but a condition of doing it at all.

Part Five is about making: a first robot of your own, and a tour of the machines that already work for a living.

Each chapter ends with something to do. Some of the exercises need only paper; a few need a phone, a ruler or a cheap motor. Where a real component is required, a substitute is offered, because nobody should be stopped by a shopping list.

PART ONE

The Machine and Its Body

CHAPTER ONE

The Machine That Acts

In 1961, a two-tonne arm was bolted to the floor of a General Motors plant in Trenton, New Jersey, and set to work lifting hot metal castings out of a die-casting machine. It did nothing clever. It moved through a sequence of positions recorded by a human operator, over and over, for years. It was called Unimate, and it was the first industrial robot.

Visitors found it disappointing. It had no eyes, no hands in any useful sense, and no idea what it was holding. It simply repeated a motion, and in doing so it removed a man from a job that was hot, dangerous and dull. That is the pattern the industry has followed ever since: the first successful robots were not intelligent, they were useful.

This volume is about the machine that made that possible, and about the same machine, sixty years later, carrying surgical instruments and driving across warehouses. The principles have not changed much. The parts have become smaller, cheaper and vastly more precise.

Five Things a Robot Must Have

Definitions of "robot" are a small industry of their own, and most of them are either too broad to be useful or too narrow to survive the next product launch. Here is a working definition, with five tests that a machine can pass or fail.

A robot has a **body** that exists in the physical world. It occupies space, has mass, and cannot be in two places at once. This sounds trivial until you compare it with a program, which can be copied a million times and run in parallel without collision.

It has **sensors** that measure something about itself or its surroundings. Even Unimate had limit switches and encoders, though no one would call them senses in the human meaning of the word.

It has **actuators** that change the world: motors, valves, heaters, speakers.

It has a **controller** that decides, from the sensor readings, what the actuators should do. The decision may be a fixed sequence, a feedback law, or a learned policy. The complexity of the decision is not part of the definition.

And it is **programmable**: the behaviour can be changed without rebuilding the machine. A washing machine has a body, sensors, actuators and a controller, and it fails this test. Its cycle is fixed in the design, and to change it you change the product. A robot arm can be taught a new task in an afternoon, which is why the same hardware can weld a car and pack biscuits.

Test	Unimate (1961)	Washing machine	Humanoid research platform
Body in the world	Yes	Yes	Yes
Sensors	Limit switches, encoders	Temperature, level, door	Cameras, IMU, force, joint encoders
Actuators	Hydraulic joints	Motor, pump, valve	Electric joints, hands
Controller	Sequence and limits	Fixed cycle	Many, layered
Programmable	Yes	No	Yes

The Loop, With Physics Attached

Volume One drew the agent as a loop: sense, decide, act. Here the same loop has consequences that software does not have, and three of them shape every design decision in this volume.

The loop takes time. A sensor reading is not instantaneous, a computation takes milliseconds, an actuator takes time to move, and the world changes while all of this happens. A robot driving at 1.5 m/s covers 15 mm in 10 ms. If the total delay from sensing to action is 50 ms, the machine is reacting to a world that is 75 mm out of date. At walking pace that is harmless. At running pace it is the difference between stopping and falling.

Mistakes are irreversible. A program that writes the wrong file can be rerun. An arm that drives into a fixture at full torque has bent something, possibly the fixture and possibly a person. This is why the vocabulary of robotics is full of words that software engineering does not use: stall, backlash, backdrive, compliance, factor of safety.

Energy has to come from somewhere. Every action consumes power, generates heat, and wears a part. A robot that runs for eight hours must carry eight hours of energy or collect it from a cable. A motor that produces the torque you asked for will get hot doing it, and a gearbox that transmits that torque will wear.

These three facts explain why robot design is a negotiation rather than an optimisation. You want it strong, fast, precise, cheap, light and safe. You can have four of those, and the choice of which four is the engineering.

How Much Freedom, How Much Help

A useful way to sort robots is by how much of the decision is theirs.

A **teleoperated** machine does what a human tells it, continuously. A surgical robot, a bomb-disposal vehicle, a drone under manual control. The human supplies the intelligence and the machine supplies precision, reach or safety.

A **semi-autonomous** machine takes instructions at a higher level and works out the details. "Go to bay 7 and pick up the pallet" is a task; the machine decides the route, the speed and the grasp. Most useful field robots are here, and this is where the interesting engineering lives, because the human and the machine must share a model of what is happening.

An **autonomous** machine is given a goal and no further help. A vacuum robot, a warehouse shuttle, an agricultural sprayer on a known field. Autonomy is narrow by nature: these machines are autonomous within a defined world, and they fail at its edges, which is why the volumes on perception, localisation and planning exist.

The categories are not a ladder of virtue. A teleoperated surgical robot is better than an autonomous one would be, because the surgeon's judgement is the point. The question is always which decisions should be delegated, and the answer depends on what happens when the machine gets it wrong.

Three Questions for Any Body

When you meet a new robot, ask three questions. The answers describe the machine more completely than any marketing page.

How strong is it, and at what distance? Strength in a robot is torque at a joint, or force at a tool, and it depends on reach. An arm that lifts 20 kg close to

its base may lift 8 kg at full extension, because torque is force times distance. The specification sheet that lists a single payload figure is telling you the best case.

How precise is it, and at what speed? Precision costs money and speed costs precision. A machine that positions to 0.02 mm will do so slowly, after settling. A machine that moves fast will overshoot, vibrate and settle late. The two figures together, with the settling time, describe the machine.

What happens when it fails? A robot with a brake that holds the arm when power is lost is a different product from one that drops it. A robot with a force limit that stops it when it touches a person is a different product again. The failure mode is a design feature, and it is chosen either deliberately or by accident.

A Family Portrait

Robots come in a small number of body plans, and each plan solves a different geometry problem.

The **industrial arm** is a chain of rigid links on a fixed base, usually six joints, designed for repeatability rather than for moving around. It is the workhorse: welding, painting, picking, packing. It is fast, accurate, and dangerous to be near when running, which is why it lives behind a fence.

The **mobile robot** moves its base. Wheels on a flat floor, tracks on rough ground, legs where wheels cannot go. Its hard problems are localisation and perception rather than reach, and it usually carries less payload than an arm because it must carry its own power.

The **drone** flies, which means it must control its orientation continuously or fall. This makes its control loop fast and its endurance short: most multirotors fly for twenty to forty minutes, and every gram of payload costs minutes.

The **humanoid** copies a body plan that evolved for climbing trees and throwing things. It is the hardest platform in the field, because balance is a continuous negotiation with gravity, and it is pursued because the world is built for human shapes: stairs, doors, tools, shelves.

The **collaborative arm** is a small industrial arm designed to work beside people without a fence, using force limits, rounded shapes and sensors that stop it on contact. It trades speed for safety, and it changed the economics of small-batch manufacturing.

Each family has its own arithmetic. Part Two of this volume gives you the arithmetic for all of them.

Problems

1. Take the five tests above and apply them to three machines you can name: a lift, a self-driving car, a robot vacuum. For each, decide whether it passes each test and write one sentence of justification.

2. A robot moves at 2 m/s. Its sensor takes 20 ms to produce a reading, its controller takes 10 ms, and its motor takes 30 ms to respond. How far does the robot travel between the moment the world changes and the moment its action reflects that change?

3. A specification sheet says an arm has a payload of 12 kg and a reach of 1.4 m. Write the two questions you would ask before believing that the arm can lift 12 kg at 1.4 m.

4. Think of a machine that is automatic but not programmable, and one that is programmable but not mobile. Explain in one sentence each why the distinction matters for the people who use them.

CHAPTER TWO

Bodies: Links, Frames and Stiffness

A robot arm looks solid. It is, in fact, a spring with a motor on the end. Load it and it bends; tap it and it rings; run it fast and it whips. The entire art of mechanical design for robots is deciding where the bending is acceptable, where it must be eliminated, and what it costs to eliminate it.

This chapter is about the bones of the machine: the links that connect one joint to the next, the frames that describe where everything is, and the two properties that decide whether a design will work — stiffness and mass. It is not a chapter about manufacturing. It is a chapter about the numbers you need before you choose a material, a wall thickness or a bearing.

A Body Is a Set of Frames

Engineers describe a robot's geometry with **frames**: coordinate systems attached to parts of the machine. There is a frame fixed to the floor (the base), a frame at the end of each link, and a frame at the tool. The position and orientation of one frame relative to another is what a robot program ultimately computes.

This is not abstraction for its own sake. A robot that picks up a part must know where the part is in the camera's frame, where the camera is in the base's frame, and where the gripper is in the base's frame. If any of those three relationships is wrong, the robot misses. Most industrial robot failures that involve "the robot went to the wrong place" are frame errors, not control errors: a camera mounted two millimetres off its nominal bracket, or a tool length entered incorrectly in the controller.

The chain of frames from base to tool is called the **kinematic chain**, and every link in the chain adds a small error. A six-axis arm with a repeatability of 0.05 mm at the tool may have a link-by-link tolerance that adds up to several tenths of a millimetre in absolute accuracy. Repeatability — returning to the

same point — is easier than accuracy — going to the right point. A machine can be very repeatable and quite inaccurate, which is fine for a welding robot working on a fixture and hopeless for a robot that must use a camera to find a part on a table.

Stiffness Is Not Strength

Two properties are constantly confused, and the confusion produces machines that break or wobble for no apparent reason.

Strength is the load at which something breaks or yields permanently. **Stiffness** is how much it deflects under a load it survives. A steel ruler is strong and floppy. A ceramic plate is stiff and brittle.

Robots usually fail on stiffness long before they fail on strength. The arm does not break; it sags, so the gripper arrives 3 mm low. It does not snap; it vibrates for 400 ms after every move, so the cycle time doubles.

The deflection of a simple cantilever — a link fixed at one end, loaded at the other — is given by

$$\text{deflection} = \text{force} \times \text{length}^3 / (3 \times E \times I)$$

where E is the stiffness of the material (its Young's modulus) and I is a property of the cross-section called the second moment of area. The important term is the cube on the length. Double the reach and the deflection goes up eightfold for the same load and section. This single fact explains why long-reach robots are expensive: they need enormous sections, or they accept poor accuracy, or they are slow.

Work an example. Take an aluminium tube 0.5 m long, 30 mm outside diameter, 2 mm wall, with a 50 N load at the tip — about 5 kg hanging off the end.

- The second moment of area for a tube is pi/64 times (outer diameter to the fourth power minus inner diameter to the fourth power). With those dimensions, I is about 1.73 x 10■■ m■.
- Aluminium has E of about 69 GPa, or 69 x 10■ N/m².
- The deflection is 50 x 0.125 / (3 x 69 x 10■ x 1.73 x 10■■), which is approximately 0.0017 m, or 1.7 mm.

A 5 kg load bends the arm by 1.7 mm. If your task needs 0.2 mm, the tube is ten times too floppy. Doubling the wall thickness does not quadruple the stiff-

ness, because I depends on the difference of fourth powers; increasing the diameter helps far more than adding material. This is why robot links are tubes and not solid bars: a tube puts material where it does the most good, away from the neutral axis.

The Enemy Is Vibration

A spring with a mass on the end has a natural frequency at which it likes to oscillate. Push it and let go, and it rings at that frequency. In a robot, that ringing is the settling time after every move, and settling time is often the largest single component of a cycle.

The natural frequency of a simple mass on a spring is

```
frequency = (1 / 2pi) x square root of (stiffness / mass)
```

For the tube above, the stiffness at the tip is about 28,700 N/m. With a 2 kg mass at the end, the frequency works out to about 19 Hz. That means the arm oscillates roughly nineteen times a second after a sudden stop, and each oscillation decays according to the damping. With light damping, the arm may still be visibly moving 300 ms later.

Three consequences follow, and all three are design decisions.

You can make the structure stiffer, which raises the frequency and shortens the ringing. This costs mass and money.

You can reduce the moving mass, especially at the far end. A gripper that weighs 1 kg instead of 3 kg does more for the dynamics than a thicker tube, because the mass is at the tip where it counts most.

You can shape the motion so that it does not excite the resonance. This is called input shaping, and it is a lovely piece of engineering: instead of moving directly to the target, the controller sends two or three impulses timed to cancel the oscillation. It costs nothing but software, and it can cut settling time by half on a machine that is already built.

Materials, and What They Cost

The choice of material is a compromise between stiffness, weight, strength, cost and how easy it is to make into the shape you want.

Material	Density (kg/m³)	Young's modulus (GPa)	Stiffness per kg (MN.m/kg)	Typical use
Steel	7,850	200	25	Bearings, screws, shafts
Aluminium	2,700	69	26	Links, brackets, frames
Titanium	4,500	110	24	High-value, weight-critical parts
Carbon fibre composite	1,600	70–150	45–90	Racing arms, drones
PLA (3D printed)	1,240	3.5	2.8	Prototypes, light covers
Nylon (3D printed)	1,140	3	2.6	Gears at low load, fixtures

Read the fourth column carefully, because it is the lesson. Steel and aluminium have almost identical stiffness per kilogram. Aluminium wins in practice not because it is stiffer per kilo but because it is easier to machine into a large tube, does not rust, and is cheaper to buy in the shapes robots need. Carbon fibre is genuinely better per kilogram, which is why it appears in drones and racing machines, and it is expensive, brittle under impact, and difficult to join.

The 3D printed plastics are an order of magnitude worse. That does not make them useless; it makes them suitable for prototypes, covers, fixtures and lightly loaded parts. A student who prints a 400 mm robot arm in PLA and wonders why it chatters has not met a mysterious problem. They have met the table above.

Mass at the End of the World

There is a rule in machine design that applies to every robot ever built: mass at the end of an arm is ten times more expensive than mass at the base.

The reason is torque. A joint must produce enough torque to hold the weight of everything beyond it, times the distance to that weight, plus the torque needed to accelerate it. Move a 1 kg gripper out by 0.3 m and you have added about 3 N.m of static torque at the wrist, permanently, in every pose. That torque has to come through the gearbox, the motor and the structure, and it appears again as heat.

The practical consequences are everywhere once you look for them. Cables are routed along the structure rather than hanging. The heaviest components — batteries, controllers, pumps — sit near the base. Tooling is made as light as the task allows. A robot that is 20% lighter at the tip can often be 20% faster on the same motors, which is a large commercial advantage for a small mechanical change.

Problems

1. A link is 0.6 m long, made of aluminium tube with a 40 mm outside diameter and 3 mm wall. Compute its second moment of area and its deflection under a 30 N tip load. Then recompute with a 2 mm wall and say how much stiffness you lost.

2. Using your answer to question 1, estimate the natural frequency with a 1.5 kg mass at the tip. What settling time would you expect if the damping is light?

3. A design uses a solid aluminium bar of 20 mm diameter instead of a tube of the same outside diameter. Which is stiffer, and why? Which is heavier, and by how much per metre?

4. Explain in two sentences why a robot can be repeatable to 0.05 mm and accurate only to 0.5 mm.

5. List four things you would do to reduce the settling time of an arm that rings for 300 ms after every move, in order of cost.

CHAPTER THREE

Joints, Degrees of Freedom and Workspace

Two robots can have the same number of joints and behave nothing alike. One can reach around a corner, one cannot; one can hold a tool at any angle, one can only point. The difference is not power or precision. It is geometry, and geometry can be counted.

This chapter gives you the counting. It is a small piece of mathematics with an unusual power: it tells you, before you build anything, how many independent motions a mechanism has, and therefore whether it can do the task you have in mind.

Two Kinds of Joint

A joint is a connection that allows some relative motion and forbids the rest. Robots use a small vocabulary of them.

A **revolute** joint allows rotation about one axis: a door hinge, an elbow, a wheel on an axle. It is by far the most common, because bearings are cheap, precise and long-lived.

A **prismatic** joint allows sliding along one axis: a drawer, a telescopic mast, a linear actuator. It is common in gantries and 3D printers, less common in arms because linear bearings are bulkier than rotary ones for the same load.

A **spherical** joint allows rotation about three axes, like a shoulder. It is hard to build with precision, so robots usually make one from three revolute joints with their axes meeting at a point, which behaves like a ball joint but can be driven.

A **cylindrical** joint allows rotation and sliding about the same axis. A **planar** joint allows two translations and one rotation. A **free** joint allows everything, which is what happens when the gripper lets go.

Each joint type allows a certain number of independent motions, and that number is called its **freedom**: one for revolute and prismatic, three for spheric-

al, and so on.

Counting Freedom

The number of independent motions a whole mechanism can make is its **degrees of freedom**, usually written DOF. A rigid body floating in space has six: three translations and three rotations. Each joint removes some of those and leaves the rest.

The standard formula for a spatial mechanism is

DOF = 6 x (number of links - 1) - sum over joints of (6 - freedom of that joint)

The subtraction of one link accounts for the ground, which is fixed. A planar mechanism, where everything moves in one plane, uses 3 instead of 6.

Test it on three examples.

A two-link planar arm. Three links (ground, upper arm, forearm) and two revolute joints. DOF = 3 x (3 - 1) - 2 x (3 - 1) = 6 - 4 = 2. Two joints, two freedoms. Obvious, and the formula agrees.

A four-bar linkage. Four links and four revolute joints, all planar. DOF = 3 x (4 - 1) - 4 x (3 - 1) = 9 - 8 = 1. Push one link and the rest follow; there is exactly one motion. This is why four-bars are used as mechanisms rather than as robots.

A six-axis industrial arm. Seven links including ground, six revolute joints. DOF = 6 x (7 - 1) - 6 x (6 - 1) = 36 - 30 = 6. Six joints, six freedoms, and the tool can be placed at any position and orientation inside the workspace. This is why six is the magic number in industrial robotics: fewer than six cannot reach every pose, and more than six is redundant, which is useful but complicated.

The formula has a famous failure. It gives the freedom of the *mechanism*, not of a given *pose*. Some poses are **singular**: configurations in which two joints line up and the mechanism loses a freedom in practice even though the count says otherwise. A six-axis arm at full stretch cannot move further outward, and a two-link arm with the elbow straight cannot move along the line of the arm. Every robot has singularities, and every robot program must avoid them. The volumes on kinematics return to this with the tools to compute them.

Serial and Parallel

The joints of most robots are arranged in a chain, one after another, from base to tool. This is a **serial** mechanism: an arm, a leg, a crane. Its advantages are reach and dexterity; its disadvantages are that every joint carries the weight of everything beyond it, and every link's flexibility adds to the error at the tip.

A **parallel** mechanism connects the base to the platform with several chains at once. The Stewart platform, used in flight simulators and some machining centres, has six legs and six degrees of freedom. Its advantages are stiffness and accuracy for a given mass, because loads are shared and errors do not stack. Its disadvantages are a small workspace and a geometry that is difficult to design around.

	Serial	Parallel
Structure	One chain, base to tool	Several chains, base to platform
Workspace	Large relative to size	Small relative to size
Stiffness	Lower; errors accumulate	Higher; loads shared
Typical use	Arms, legs, cranes	Simulators, precision stages, some machine tools

The choice is a geometry decision, not a fashion. A robot that must reach into a car body needs the workspace of a serial arm. A machine that must hold a tool rigidly to a micron needs the stiffness of a parallel one.

Reach and Workspace

The **workspace** is the set of all positions the tool can reach. For a two-link planar arm with lengths L1 and L2, it is the region between two circles: an outer radius of L1 + L2, an inner radius of the absolute difference between them. If the links are 0.4 m and 0.3 m, the tool can reach anything from 0.1 m to 0.7 m from the base, inside the swept area.

That inner hole is not a technicality. It means the arm cannot reach its own shoulder, which is fine for a welding robot and fatal for a machine that must place a part at the base. Designers fill the hole with a seventh joint, a different shoulder geometry, or a turning table that brings the work to the robot.

For a six-axis arm the workspace is a complicated, doughnut-like volume that is usually published as a diagram in the manual. Two numbers matter more

than the shape: the maximum reach, and the payload at maximum reach. The first is geometry. The second is torque, and it is the subject of the next part of this volume.

Wrist, Singularity and the Sixth Joint

In a six-axis arm, the first three joints usually decide where the tool is, and the last three decide which way it points. The last three are called the wrist, and in most industrial arms their axes meet at a single point, which makes the mathematics of orientation tractable and the mechanics of the wrist compact.

A spherical wrist has a characteristic weakness, and it is worth understanding because it causes real production problems. When the second wrist axis lines up with the first, two of the three rotations do the same thing, and the arm loses a degree of freedom in practice. The controller must then rotate one joint extremely fast to achieve a small change in orientation, or it must refuse the motion. Machinists call the region around this configuration a singularity, and programmers learn to keep the tool away from it.

The consequences are measurable. A wrist error of one degree, on a tool 0.5 m from the wrist centre, moves the tool by 0.5 x 0.0175 = 8.7 mm. Near a singularity, the geometry amplifies small commands into large joint motions, and the error grows the same way. A program that runs perfectly at the centre of the workspace can produce a wild swing at the edge of it, and the same program can behave differently on two machines of the same model if their zero offsets differ by a fraction of a degree.

The sixth joint has one more job that is easy to overlook: it sets the roll of the tool. A gripper that must approach a part from the side needs the roll to be free regardless of how the arm is positioned, and a task that ignores roll will work in half the reachable space and fail mysteriously in the other half. When a robot program behaves inconsistently across a workspace, the first thing to check is not the control loop but the orientation of the tool frame.

Problems

1. Count the degrees of freedom of a planar mechanism with five links and five revolute joints. Then draw one and check whether the count matches your intuition.

2. A three-link planar arm has links of 0.3 m, 0.25 m and 0.15 m. What is the outer radius of its workspace and what is the inner radius?

3. A six-axis arm has six joints. Explain in one sentence why a task that only needs position, not orientation, might still use six.

4. A robot must place a component at a point 50 mm from the centre of its own base. Its two-link arm has links of 0.35 m and 0.30 m, so the inner radius is 0.05 m. Is the task possible? What would you change?

5. Find a picture of an industrial robot and count its joints. Then decide whether it is serial or parallel, and what its workspace probably looks like.

PART TWO

Muscle: Motors, Gears and Power

CHAPTER FOUR

Electric Motors

Almost every robot you will meet is moved by an electric motor, and almost every motor is chosen badly the first time. The reason is not ignorance of electromagnetism. It is that the two numbers people look at — power and voltage — are the two least useful numbers on the label.

What matters is torque at a given speed, and the relationship between them. A motor that produces 20 watts at 3,000 revolutions per minute and one that produces 20 watts at 30 revolutions per minute are completely different animals, and neither will do the other's job.

Two Numbers, One Curve

A motor produces **torque**, a turning force, measured in newton-metres. It turns at a **speed**, measured in radians per second (or revolutions per minute, which is the same thing divided by 9.55). Multiply the two and you get mechanical power in watts.

The crucial fact is that a motor cannot give you both maximum torque and maximum speed at once. At zero speed it produces its largest torque, called the **stall torque**. At zero load it spins at its fastest, called the **no-load speed**. Between those two points the relationship is roughly a straight line:

```
torque available falls as speed rises
```

This curve is the single most useful document about any motor, and it is the first thing to ask a supplier for. A motor with a high stall torque and a low top speed is a strong, slow motor. A motor with the opposite is fast and weak. Neither is better; they are different trades.

What the Motor Feels

Inside a brushed direct-current motor, the physics is simple enough to state in three sentences, and those three sentences explain almost every behaviour you will see.

The **speed** of the motor is approximately proportional to the voltage applied, minus the losses. Double the voltage and the motor tries to spin twice as fast.

The **torque** is proportional to the current flowing through the windings. Demand more torque by loading the shaft, and the motor draws more current to supply it.

The winding has **resistance**. Current through resistance produces heat, and heat is what destroys motors.

Put those together and you get the behaviour that surprises beginners. When you stall a motor — hold the shaft so it cannot turn — the current is limited only by the winding resistance. A small motor with a 1 Ω winding on a 24 V supply will draw 24 A at stall, which is 576 W of heat inside a component the size of a matchbox. It will not survive for many seconds.

That is why a stalled robot arm can burn out a motor in the time it takes to walk across the room, and why the first protective measure in any motorised design is a current limit rather than a bigger motor.

Choosing a Motor from the Task

Here is the calculation that turns a requirement into a purchase order. It has four steps and no calculus.

Step 1: find the torque the joint must hold. A link of 0.5 m carrying a 1 kg mass at the tip needs a torque of mass x gravity x distance: 1 x 9.81 x 0.5, which is about 4.9 N.m, just to hold it still against gravity. Add the torque to accelerate it: a modest 1 rad/s² of angular acceleration with an inertia of about 0.08 kg.m² needs another 0.08 N.m. Round up, and design for 6 N.m at the joint.

Step 2: choose a gear ratio. Small motors produce small torques; a typical hobby motor might produce 0.1 N.m of continuous torque. To get 6 N.m you need a ratio of at least 60:1, and that is before losses. With a gearbox that is 80% efficient, you need 7.5 N.m at the gearbox output, so the motor must supply 7.5 / 60 = 0.125 N.m. Choose a motor rated a little above that, say 0.15 N.m

continuous.

Step 3: check the speed. The same gearbox divides speed by 60. If the motor spins at 3,000 revolutions per minute, or 314 rad/s, the joint turns at 5.2 rad/s — about 300 degrees per second. If your task needs the arm to move 90 degrees in 0.5 s, that is 3.1 rad/s, and the design passes with room to spare.

Step 4: check the power and the heat. The motor is delivering 0.15 N.m at 314 rad/s, which is about 47 W of mechanical power. At 80% efficiency in the gearbox and perhaps 75% in the motor, it is drawing around 78 W of electrical power, most of which becomes heat somewhere. That heat has to leave a small metal cylinder. Chapter 6 does the arithmetic.

Quantity	Symbol	Formula	Example
Joint torque needed	T	m x g x r	1 x 9.81 x 0.5 = 4.9 N.m
Gear ratio	N	T_joint / (T_motor x efficiency)	6 / (0.1 x 0.8) = 75
Joint speed	ω	motor speed / N	314 / 75 = 4.2 rad/s
Motor power	P	T x ω	0.15 x 314 = 47 W

The four steps take ten minutes and prevent the most common failure in student robotics: a beautiful mechanism driven by a motor that cannot hold it.

Steppers, Servos and Brushless Motors

Three families dominate small and medium robots, and each has a characteristic strength.

A **stepper motor** moves in fixed increments — typically 1.8 degrees per step, or 200 steps per revolution — and holds its position when stationary. It needs no sensor to know where it is, which is why it appears in 3D printers and cheap Cartesian robots. It loses steps silently if overloaded, and it draws current even when holding still.

A **hobby servo** contains a small motor, a gearbox and a position sensor in one box, and accepts a command in the form of a pulse width: about 1 ms for one extreme, 2 ms for the other, repeated 50 times a second. It is the fastest route to a moving joint, and it is limited in torque, accuracy and life.

A **brushless motor** replaces the mechanical commutator with electronics, which makes it more efficient, longer-lived and capable of higher power for its size. It requires a driver that knows the rotor position, which adds cost and complexity. Almost every serious robot arm, drone and electric vehicle uses them.

	Stepper	Hobby servo	Brushless
Position feedback	None needed	Internal	Required (Hall or encoder)
Torque for size	Moderate	Low	High
Holding torque	Yes, with current	Yes	With control
Typical use	3D printers, pick-and-place	Small joints, steering	Drones, arms, vehicles
Main risk	Silent loss of steps	Gear wear, limited life	Driver complexity, cost

Problems

1. A motor has a stall torque of 0.8 N.m and a no-load speed of 4,000 revolutions per minute. Assuming a straight-line torque-speed curve, what torque does it produce at 2,000 revolutions per minute? What mechanical power is that?

2. A joint must hold 3 kg at 0.4 m. Compute the torque required. With a gearbox of 40:1 and 85% efficiency, what continuous motor torque is needed?

3. A 12 V motor has a winding resistance of 2 Ω. What current flows at stall, and how much heat is produced in the winding? Why does this matter for a robot that might jam?

4. A stepper motor has 200 steps per revolution and is driven with 16x microstepping. How many commands per revolution? If a belt pulley of 20 mm diameter is on the shaft, how far does the belt move per microstep?

5. Explain in three sentences why a motor's power rating alone cannot tell you whether it will lift your arm.

CHAPTER FIVE

Gearboxes, Belts and Screws

A motor on its own is fast and weak. A robot needs slow and strong, which means something must trade speed for torque. That something is a transmission, and it is where most of the mechanical personality of a robot comes from: how precisely it moves, how loudly it complains, how long it lasts, and how much it costs.

The physics of a transmission is one sentence long: what you gain in torque you pay for in speed, plus a tax called efficiency. The engineering is everything else.

Gears Do Two Things

A pair of gears with 10 teeth driving a gear with 50 teeth has a **ratio** of 5:1. The output turns five times more slowly than the input, and produces five times the torque, minus losses.

That is the whole trade. If the motor has 0.1 N.m at 3,000 revolutions per minute, the output has 0.5 N.m at 600 revolutions per minute. Power is unchanged in theory — torque times speed is the same on both sides — which is a useful check on any gear calculation. If your numbers show more power out than in, you have made an arithmetic error or invented a perpetual motion machine.

Two details matter more than the ratio.

Efficiency. Every stage loses some power to friction, and the losses compound. A single spur gear pair is about 97% efficient. A planetary gearbox is typically 90% for one stage and 80% for two. A worm drive is often 50% or worse, which is why it is used where you want the load to hold itself rather than where you want efficiency.

Backlash. The small gap between teeth, needed so that gears do not jam, means the output can move a little without the input moving at all. In a cheap

gearbox, backlash might be 1 degree. On an arm 0.5 m long, 1 degree of backlash is 8.7 mm of free movement at the tip, in every direction change. A robot that must draw a straight line will produce a wavy one, and no amount of encoder resolution will fix it, because the error is in the mechanism, between the sensor and the tool.

Transmission	Typical ratio per stage	Efficiency	Backlash	Used for
Spur gears	3:1 to 6:1	97%	Moderate	Simple drives, printers
Planetary	3:1 to 10:1	90%	Low	Robot joints, hubs
Worm drive	10:1 to 100:1	40–70%	Moderate	Holding loads, lifts
Timing belt	1:1 to 5:1	96%	Very low	Gantries, mobile robots
Lead screw	—	30–50%	Low	Linear motion, low duty
Ball screw	—	90%	Very low	Linear motion, precision
Harmonic drive	30:1 to 160:1	70–85%	Near zero	Robot arm joints

Belts and Screws

Not every transmission is a gearbox, and two alternatives deserve their own paragraph because they solve problems gears cannot.

A **timing belt** transfers motion over a distance, which is exactly what a gantry or a mobile robot needs. It is cheap, quiet, tolerates misalignment and has almost no backlash if tensioned properly. Its weakness is stretch under load, which limits precision, and its need for maintenance as it ages.

A **lead screw** turns rotation into straight-line motion. One turn advances the nut by one **lead**, so the relationship is geometric rather than mysterious. The force it can produce is

```
force = 2pi x torque x efficiency / lead
```

Take a screw with a 5 mm lead, a torque of 0.5 N.m and an efficiency of 90%. The force is 2pi x 0.5 x 0.9 / 0.005, which is about 565 N — enough to lift 57 kg. The same screw in a cheap ACME version at 40% efficiency gives

about 250 N. The difference between a lead screw and a ball screw is mostly this efficiency, and it shows up as heat, noise and wear.

The trade for screws is speed: to move 200 mm at a 5 mm lead takes 40 turns. A motor at 3,000 revolutions per minute does that in 0.8 s, which is fine for a lift and slow for a pick-and-place axis.

Zero Backlash and Why It Costs

For a robot arm that must follow a curved path while a camera watches, backlash is fatal. The mechanism must have as little free play as possible, and the two devices that achieve this are both expensive and both worth understanding.

A **harmonic drive** uses a flexible cup with external teeth that meshes with a rigid ring with internal teeth, two teeth fewer. A wave generator inside deforms the cup so that it engages at two points, and each turn of the input advances the output by two teeth. A ratio of 100:1 is normal, the backlash is effectively zero, and the device is compact and light for its torque. It is in the joints of many industrial and humanoid robots.

A **cycloidal drive** uses a similar principle with a lobed disc rolling inside a ring of pins. It handles shock loads better than a harmonic drive and is common in the base joints of heavy robots.

Both cost more than a planetary gearbox, and both repay it in a machine that can move accurately in both directions. When you see the price of a collaborative robot arm and wonder why it is not cheaper, look at the six harmonic drives in its joints.

Choosing a Transmission

The decision comes down to five questions, and they can be answered on one page.

How much torque must be transmitted, and how fast must the output move? This fixes the ratio.

How much backlash can the task tolerate? A welding robot working on a fixture can tolerate a lot. A robot drawing with a pen cannot.

How efficient must it be? Battery-powered machines care, because loss becomes heat and heat becomes shorter runtime.

How long must it last, and how will it be maintained? Grease dries, belts stretch, gears wear.

What does it cost, and can you buy it? A custom gearbox is a project; a standard one is a purchase order.

A Gearbox for a Four-Kilogram Arm

Work the whole selection once, because the arithmetic is what turns a catalogue into a decision.

The task: lift a 4 kg payload at 0.45 m from the joint, move it through 90 degrees in 0.8 s, and hold it still while a camera inspects it.

Torque to hold. Four kilograms at 0.45 m gives 4 x 9.81 x 0.45 = 17.7 N.m. Add the mass of the link itself, say 1.5 kg with its centre of mass at 0.25 m: another 3.7 N.m. Total static torque: 21.4 N.m.

Torque to accelerate. The inertia of the payload about the joint is $4 \times 0.45^2 = 0.81$ kg.m², and the link contributes perhaps 0.03 kg.m². For 90 degrees (1.57 rad) in 0.8 s with a smooth profile, the peak acceleration is roughly $4 \times 1.57 / 0.8^2 = 9.8$ rad/s². Torque is inertia times acceleration: 0.84 x 9.8 = 8.2 N.m.

Total design torque. 21.4 + 8.2 = 29.6 N.m. Add a margin of 50% for friction, misalignment and the fact that the payload may be heavier than the label: design for 45 N.m.

Gearbox. A 50:1 planetary gearbox with 85% efficiency requires 45 / (50 x 0.85) = 1.06 N.m at the motor. That is a substantial motor — a 200 W class brushless with a rated torque near 1 N.m — and it is the reason serious arms are expensive.

Speed. The motor must turn at 50 times the joint speed. The joint needs a peak of about 4 rad/s, so the motor needs 200 rad/s, or about 1,900 revolutions per minute. A motor rated at 3,000 revolutions per minute has headroom.

Power and heat. At 1.06 N.m and 200 rad/s the motor delivers 212 W. At 80% motor efficiency it draws 265 W and produces about 53 W of heat. A motor that size can dissipate 53 W continuously if it is mounted on a metal structure, and not if it is wrapped in plastic.

Stage	Quantity	Value
Static torque	m x g x r, summed	21.4 N.m
Acceleration torque	I x α	8.2 N.m

Stage	Quantity	Value
Design torque with margin	x 1.5	45 N.m
Motor torque needed	design / (ratio x efficiency)	1.06 N.m
Motor speed needed	joint speed x ratio	200 rad/s

If the arithmetic produces a motor you cannot afford, the design changes rather than the physics. Shorten the link, reduce the payload, add a counterbalance spring, or move the heavy parts of the tool closer to the wrist. Each of those is a design decision, and each is cheaper than buying a motor twice the size.

Problems

1. A motor produces 0.12 N.m at 3,000 revolutions per minute. A 30:1 planetary gearbox with 90% efficiency is attached. Compute the output torque and output speed. Verify that the power is roughly preserved minus losses.

2. A gearbox has 1.5 degrees of backlash. How far does the tip of a 0.6 m arm move without the motor turning? What task would this ruin, and what task would not notice?

3. A lead screw has a lead of 8 mm and an efficiency of 45%. With 0.4 N.m of torque, what force can it produce? How many turns are needed to move 120 mm?

4. A machine needs 15 N.m at 1 rad/s from a 0.2 N.m motor. What ratio do you need, and which transmission type from the table would you choose? Justify the choice in two sentences.

5. Explain why a belt drive can have almost no backlash and still be less precise than a gear train.

CHAPTER SIX

Power: Batteries, Wires and Heat

Every robot carries its own physics. The motors demand current, the wires resist it, the battery delivers what it can, and the difference between the three becomes heat. A design that ignores this arithmetic works beautifully on the bench for ten minutes and then behaves oddly, resets itself, or smells.

This chapter is about the three numbers that keep a robot alive: energy, current and temperature. It contains more arithmetic than any other chapter in the volume, and it is the one to read with a calculator.

Energy Is What You Are Buying

A battery is a tank, and the size of the tank is measured in **watt-hours**: the power it can supply multiplied by the time it can supply it.

A pack labelled 11.1 V and 5 Ah holds 11.1 x 5 = 55.5 Wh. If the robot draws 5 A on average, that is 55.5 W, and the pack lasts about an hour in theory. In practice, perhaps 45 minutes, because batteries lose capacity at high discharge rates, the voltage sags as they empty, and running a pack flat shortens its life.

Work a realistic case. A small mobile robot has two drive motors drawing 3 A each when moving, a computer drawing 1.5 A, and sensors drawing 0.5 A. On a 22.2 V pack, the total current is 8 A, so the power is 178 W. For two hours of operation, the energy needed is 355 Wh, which means a pack of about 16 Ah at 22.2 V. That pack weighs roughly 1.5 kg and costs more than the motors.

Notice how quickly the numbers grow. A robot that must run all day either carries an enormous battery, docks and recharges, or follows a cable. There is no fourth option, which is why warehouse robots have charging stations in the aisles.

The Three Numbers on a Battery

Every pack has three ratings, and all three matter.

Voltage sets the speed of the motors and must match the driver. A 24 V motor on a 12 V pack runs at about half speed and never reaches its rated power.

Capacity in amp-hours sets the runtime, as above. It is a rating at a particular discharge rate; a pack that gives 5 Ah at a gentle drain may give 4 Ah at a brutal one.

C-rate sets the maximum current. A 5 Ah pack rated at 20C can deliver 100 A briefly. Exceed it and the pack heats, sags, and in the case of lithium polymers, may ignite.

Chemistry	Energy per kg	Cycle life	Notes
Lead-acid	30–40 Wh/kg	300–500	Cheap, heavy, tolerant of abuse
Nickel-metal hydride	60–120 Wh/kg	500–1,000	Robust, bulky, older technology
Lithium-ion	150–250 Wh/kg	500–2,000	The default for robots, needs a battery manager
Lithium polymer	150–250 Wh/kg	300–1,000	Same chemistry in a pouch, high current, fire risk

Lithium packs require a **battery management system** that prevents overcharge, over-discharge and cell imbalance. A pack without one is a fire waiting for a reason. This is not a cautious aside; it is the single most common cause of serious damage in student robotics, and it happens when a pack is charged on the wrong setting, punctured, or discharged below its minimum.

Wires Are Resistors

A wire is a resistor with good manners. It has a small resistance per metre, and when current flows through it the loss is given by

$$\text{power lost} = \text{current}^2 \times \text{resistance}$$

The square is the reason wire sizing surprises people. Doubling the current quadruples the loss. A wire that runs cool at 5 A can melt its insulation at 20 A, and it does so while the robot appears to work.

Take a copper wire of 0.75 mm² cross-section, which has a resistance of about 23 milliohms per metre. A drive motor 2 m from the battery needs 4 m of wire there and back, so 0.092 Ω. At 20 A, the loss is 400 x 0.092 = 37 W, spread along the wire, and the voltage drop is 20 x 0.092 = 1.84 V. On a 24 V system that is nearly 8% of the supply lost before the motor sees any of it.

Two consequences follow. Use thicker wire for high-current paths, and keep those paths short by putting the battery near the drivers. Both are free at the design stage and expensive afterwards.

Current	Minimum sensible cross-section	Typical use
Up to 2 A	0.2 mm²	Sensors, signals, small fans
2–5 A	0.5 mm²	Small motors, electronics
5–15 A	1.0 mm²	Drive motors on a small robot
15–40 A	2.5–4 mm²	Larger drives, robot arms
Above 40 A	6 mm² and up	Traction, high-power arms

Fuses, Contactors and the Stop Button

A robot with a battery and no fuse has a single fault away from a fire. The fuse is not there to protect the electronics; it is there to protect the wire, which will otherwise glow before the electronics notice.

Protection is layered, and each layer does a different job.

The **fuse or breaker** limits the current a fault can draw, and it must be sized above the normal operating current and below the wire's rating.

The **contactor or main relay** allows the whole machine to be de-energised by a single command, which is what an emergency stop must do. A proper emergency stop cuts power to the actuators directly, not by asking the software nicely. Software can hang; a contactor cannot.

The **current limit in the driver** protects the motor from a stall that the fuse is too slow to catch.

The **thermal cut-out** protects the motor when it is working hard but not stalling — the slow case that fuses never see.

An emergency stop is a safety device and must be designed as one. It should be latching, so that restarting is a deliberate act; it should be reachable from where a person would stand; and it should be tested before the robot is used in

front of anyone.

Heat: The Quiet Killer

A motor converts electrical power into mechanical power and heat. The heat has to leave through the motor's surface, and the surface is small.

Return to the arm motor from the previous chapter, which needed 0.15 N.m at 314 rad/s. That is 47 W of mechanical output. At about 75% efficiency, the motor draws 63 W and produces 16 W of heat. A small motor has a thermal resistance to the surroundings of perhaps 3 K per watt, so 16 W gives a temperature rise of about 48 °C above ambient. In a 25 °C room, the winding reaches 73 °C — uncomfortable but survivable.

Now hold the arm against a stop, so the motor stalls at 24 A. The heat in the winding is hundreds of watts, the rise is hundreds of degrees, and the insulation fails in seconds. The fuse may not blow, because the current is within its rating for a short time. The encoder may survive; the motor will not.

The design response is a **duty cycle**: the fraction of time a machine may run at a given load. A motor rated for 50% duty can deliver its rated torque half the time and must rest the other half. A robot that lifts a heavy part once every ten seconds can use a motor that would overheat if it lifted continuously, and the datasheet will say so if you read past the first page.

Problems

1. A pack is rated 14.8 V and 4 Ah. How much energy does it hold in watt-hours? If the robot draws 6 A, how long will it last in theory, and what would you expect in practice?

2. A wire of 1.0 mm² has a resistance of about 18 milliohms per metre. A motor draws 12 A through 3 m of wire (6 m round trip). Compute the power lost and the voltage drop. What fraction of a 24 V supply is lost?

3. Your robot needs 120 Wh per hour of operation and must run for four hours. What battery capacity in amp-hours do you need on a 22.2 V pack, and what mass would you expect?

4. A motor produces 30 W of heat and has a thermal resistance of 4 K/W. What temperature rise does it see? If the insulation is rated to 130 °C, what ambient temperature would make the design unsafe?

5. Write the four layers of electrical protection for a robot and, for each, the failure it prevents.

CHAPTER SEVEN

Electronics and Motor Drivers

A microcontroller is a small computer with pins. Some pins listen, some pins talk, and some pins can turn a voltage on and off. None of them can drive a motor, and the first lesson of robot electronics is understanding exactly why.

A typical microcontroller pin can supply about 20 milliamps. A small robot motor needs 2 amps, and a larger one needs 20. The ratio is a hundred to a thousand, and the gap is bridged by a driver: a component whose job is to take a small, polite signal and use it to switch a large, rude current.

Sensors In, Power Out

The architecture of almost every small robot is the same. Sensors produce voltages that the microcontroller reads. The microcontroller computes. It sends low-power commands to drivers. The drivers switch power from the battery to the motors. Anything else — displays, radios, extra memory — hangs off the same pattern.

This separation exists for three reasons. The microcontroller is fragile and expensive; the driver is robust and cheap, and it is the part you want to fail. The motor generates electrical noise that would destroy a processor sharing its supply. And the current paths are different: signals flow in milliamps along thin wires, power flows in amps along thick ones, and mixing the two produces a machine that resets whenever it drives forward.

The last of these is worth a sentence of its own. A robot whose controller reboots when the motors start is not haunted. It has a wiring problem: the motor's current pulse is dropping the supply voltage below the controller's minimum, or the noise is coupling into a signal line. The fix is layout and capacitance, not software.

The H-Bridge

A direct-current motor turns one way when voltage is applied one way and the other way when the polarity is reversed. To reverse it under software control, you need four switches arranged around the motor, and that arrangement is called an **H-bridge**.

Label the switches top-left and bottom-left, top-right and bottom-right. Close top-left and bottom-right: current flows one way and the motor turns forward. Close the other diagonal: it turns backward. Close both bottom switches: the motor is shorted across itself and brakes hard.

There is one combination that must never happen: both switches on the same side closed at once, which connects the supply directly to ground through two transistors. This is called **shoot-through**, and it destroys drivers in microseconds. Every competent driver includes a small delay, called dead time, to make shoot-through impossible even if the software makes a mistake.

The H-bridge is a good example of a general principle in this volume: the electronics exist to make the software's mistakes survivable. A microcontroller that asks for an impossible combination should get a safe outcome, not a fire.

PWM and the Average Voltage

A digital output has two states: on and off. A motor wants a smooth, adjustable voltage. The trick that bridges them is **pulse width modulation**, which switches the output on and off very fast and relies on the motor's own inertia to smooth the result.

If a 12 V supply is switched on for 30% of each cycle and off for 70%, the motor sees an average of 3.6 V. Change the duty cycle to 80% and it sees 9.6 V. The switching frequency is typically 10 to 30 kHz, above the range that produces audible whine, and the motor's inductance keeps the current continuous.

Two numbers matter in practice. The **frequency** must be high enough to keep the current smooth, and low enough that switching losses in the driver stay small. The **resolution** of the duty cycle sets how finely the speed can be controlled: 8-bit control gives 256 levels, which on a 24 V system is a step of about 94 mV.

PWM is used for more than motors. Heating elements, lights, fans, solenoid valves and even some sensors are driven this way. Whenever you see a digital

system controlling a continuous quantity cheaply, PWM is usually underneath.

Current Sensing and Motor Protection

A driver that cannot measure current cannot protect anything. Two methods are common.

A **shunt resistor** in the current path produces a small voltage proportional to the current. A 10 milliohm shunt carrying 10 A drops 0.1 V, which is easy to measure and costs 1 W of heat.

Hall sensors measure the magnetic field produced by the current and need no shunt, at higher cost.

With current measurement, the driver can implement a **current limit**: when the measured current exceeds a threshold, reduce the duty cycle or switch off for the rest of the cycle. This is what saves a motor when the arm jams against a table. It is also what allows a collaborative robot to detect a human contact: the sudden rise in current, filtered and interpreted, is the signal that something unexpected has happened.

The limit must be set from the motor's ratings, not from the driver's capability. A driver that can pass 40 A will happily pass 40 A into a motor rated for 5 A until the winding fails.

The Microcontroller's Real Constraints

Three properties of small computers shape what a robot can do, and they are worth stating plainly because they surprise people who have only written desktop software.

Time is not guaranteed. A desktop operating system schedules tasks when it feels like it. A microcontroller running a bare-metal loop executes your code in order, which is predictable, but any interrupt can delay the main loop. If a control loop must run every 1 ms, the designer must account for the worst case, not the average.

Analogue inputs are coarse. A typical 10-bit analogue-to-digital converter divides a 5 V range into 1,024 steps, which is about 4.9 mV per step. A sensor with a 0.01 V signal change per degree can therefore resolve about two degrees, no matter how good the sensor is.

Memory is small. A microcontroller with 32 kB of program memory cannot hold a camera image, a map, or a neural network of any size. Those functions move to a more capable computer, and the two communicate, which introduces the latency and failure modes of a network link inside the robot.

A Wiring Plan That Survives

Most faults in a small robot are wiring faults, and most wiring faults come from decisions made in a hurry. A plan drawn before the first crimp prevents them.

Separate the two worlds. Power wiring — battery, switch, fuse, drivers, motors — runs on one side of the machine in thick wire. Signal wiring — sensors, encoders, communication — runs on the other side in thin wire. Where they must cross, they cross at right angles.

Ground at one point. The battery negative is the star point. Every return connects to it directly, or through a bus bar near it. Daisy-chaining grounds through several boards puts the motor current through the sensor's reference, and the result is a reading that changes whenever the robot accelerates.

Fuse every branch. The main fuse protects the battery and the harness; each driver branch has its own fuse sized to the wire, not to the motor's peak current. A 5 A branch on 0.5 mm² wire is sensible; a 20 A fuse on the same wire is a fire.

Strain-relieve everything. Every wire that leaves a connector should be supported within 50 mm, so that vibration is taken by the support rather than the crimp. A robot that fails after a week usually fails at a connector that was flexed a thousand times.

Label both ends. A strip of tape with a number, written at both ends of every wire, saves an hour every time something goes wrong. It costs ten minutes.

Wire colour	Use
Red	Positive supply, after the switch
Black	Ground return
Yellow	Motor phase A
Blue	Motor phase B
Green	Sensor signal

Wire colour	Use
White	Communication

The colours are a convention, not a standard, and any convention is better than none. The machine you repair at midnight will be the machine you wired at noon, and you will not remember which black wire was which.

Problems

1. A 24 V supply drives a motor through PWM at 25% duty. What average voltage does the motor see? What average voltage at 90%?

2. A 10 milliohm shunt carries 8 A. What voltage appears across it, and how much power does it dissipate?

3. A 10-bit analogue-to-digital converter measures 0 to 5 V. How many volts per step? A temperature sensor produces 10 mV per degree; what is the smallest temperature change the system can resolve?

4. Explain in two sentences why an H-bridge needs dead time, and what happens without it.

5. A controller resets whenever the drive motors start. List three likely causes in order of probability, and the test that would distinguish them.

PART THREE

Sense: Instruments and Signals

CHAPTER EIGHT

Sensors: Turning the World into Numbers

A robot knows nothing except what its instruments tell it, and every instrument lies a little. A wheel encoder under-reports when the wheel slips. A distance sensor reports the nearest reflective surface, which may be a chair leg or a hanging coat rather than a wall. A gyroscope accumulates a small error into a large one. The skill of sensing is not finding an instrument that is always right; it is knowing how each one is wrong, and by how much.

Four Questions About Any Sensor

Specifications are written in a private language, and four words in that language decide whether a sensor is suitable.

Range. The span of values it can measure. A distance sensor that works from 0.2 m to 4 m is blind to a wall at 10 cm, which matters when the robot is parking.

Resolution. The smallest change it can report. A 12-bit encoder gives 4,096 counts, so its resolution is one part in 4,096, whatever the physical quantity.

Accuracy. How close the reported value is to the truth. A sensor can have fine resolution and poor accuracy: it reports 1.002, 1.004, 1.006 when the truth is 1.010. The numbers are precise and the answer is wrong.

Drift. How much the error changes over time and temperature. A cheap gyroscope may drift by 0.1 degrees per second, which is 6 degrees a minute and 360 degrees an hour. On a robot that integrates that signal to estimate heading, drift is not a curiosity; it is the whole problem.

There are two more properties that never appear on the datasheet and always appear in practice: **latency**, the delay between the event and the report, and **failure behaviour**, what the sensor outputs when it is broken. A sensor that fails to zero is very different from one that fails to its last value, and a robot that cannot tell the difference will happily drive on a frozen reading.

Measuring the Robot Itself

Most robot control depends on knowing the position of its own joints, and three devices do that work.

A **potentiometer** is a variable resistor that reports angle as a voltage. It is cheap, simple, and wears out, because a wiper slides on a track millions of times. It is fine for a hobby arm and unacceptable for a machine that must run for years.

An **incremental encoder** produces pulses as a disc with slots turns between a light and a sensor. Two channels, offset by a quarter of a slot, allow the controller to know both the amount and the direction of rotation. It has no absolute reference: after a power cycle it knows how far it has moved, not where it is.

An **absolute encoder** reports the full angle at every reading, using a coded disc or a magnetic pattern. It costs more and removes the need to home the joint at start-up, which is why industrial robots use them.

The arithmetic is worth doing once. An encoder with 1,000 cycles per revolution, read in quadrature, gives 4,000 counts per revolution. On a wheel of 50 mm radius, the circumference is 2pi x 0.05 = 0.314 m, so each count is 0.314 / 4,000 = 0.0785 mm of travel. That is the finest distance the robot can resolve from its wheels — and only if the wheel does not slip.

Measuring the World

The instruments that look outward are more varied, and each has a characteristic failure.

An **ultrasonic rangefinder** emits a burst of sound at about 40 kHz and times the echo. The distance is the speed of sound times the time, divided by two, because the pulse travels out and back. At 343 m/s, a wall at 1 m produces an echo after 5.8 ms. Soft surfaces absorb the sound and disappear; angled surfaces reflect it away; two robots using the same frequency interfere with each other.

An **infrared proximity sensor** measures reflected light. It is cheap, fast and fooled by colour: a black surface returns almost nothing, and a white one at the same distance looks close. It is a good cliff detector and a poor distance meter.

A **laser scanner**, or lidar, sweeps a beam and measures time of flight or phase. It gives thousands of accurate ranges per second over a wide angle,

which is why it is the backbone of mobile robot navigation. It is expensive, it dislikes rain and dust, and a black or mirrored surface can vanish.

A **camera** measures light, and everything else is inference. Its great strength is that it captures a whole scene at once; its weakness is that the mapping from pixels to metres requires calibration, and the calibration drifts when the lens is knocked.

A **force or torque sensor** measures the mechanical interaction, which is essential for any task where contact is the point: assembly, polishing, handover to a person. A **microswitch** is the crudest force sensor and remains the most reliable: it reports contact, and it does so for a few cents.

Sensor	Measures	Typical resolution	Characteristic failure
Incremental encoder	Rotation	0.09° at 4,000 counts	Loses position at power-off
Potentiometer	Angle	0.5°	Wear, noise, limited life
Ultrasonic	Distance	5 mm	Soft or angled surfaces
Infrared proximity	Presence, rough distance	Not a distance	Surface colour
Lidar	Distance, many points	10–30 mm	Rain, dust, black surfaces
Camera	Light	Depends on calibration	Lighting change, motion blur
IMU	Acceleration, rotation rate	Fine, but drifting	Bias drift over time

The Instrument That Forgets

The inertial measurement unit deserves a section of its own, because it is the clearest example of a sensor whose errors compound.

An IMU contains accelerometers that measure acceleration and gyroscopes that measure angular rate. To get orientation, you integrate the rate: angle is the sum of rate multiplied by time. To get position, you integrate acceleration twice.

Each integration turns a small error into a large one. A gyroscope with a bias of 0.1 degrees per second accumulates 6 degrees a minute. An accelerometer

with a bias of 0.01 m/s² produces a velocity error of 0.01 m/s after a second, a position error of 5 mm after one second, and 1.8 m after a minute, before any vibration is counted.

This is why no serious robot navigates on inertial data alone. The IMU is fast, smooth and locally accurate, and it is combined with a slower absolute reference — wheels, a laser scan, a camera, a satellite fix — so that the drift is corrected before it grows. That combination is the subject of the volumes on perception and localisation, and the arithmetic you have just done is the reason they exist.

Problems

1. An encoder gives 2,000 counts per revolution. A wheel of 80 mm radius is attached. How far does the robot move per count?

2. An ultrasonic sensor measures an echo after 11.6 ms. How far away is the surface? What would the reading be if the surface were a curtain?

3. A gyroscope drifts by 0.05 degrees per second. How much heading error accumulates in 5 minutes? What would you add to the robot to correct it?

4. A camera is knocked sideways by 2 mm. Explain in two sentences why the robot's grasp may now miss by more than 2 mm.

5. Choose a sensor for each of these tasks and say what would make it fail: detecting a table edge, measuring the force of a gripper, finding a person in a corridor, knowing how far a wheel has turned.

CHAPTER NINE

Signals, Noise and Sampling

A sensor reading is never a number. It is a number with a history: it was produced by a physical process, converted by electronics, sampled at intervals, and delivered with some of the world's randomness attached. The difference between a robot that works and one that twitches is usually not the sensor but what happens to its output between the sensor and the decision.

This chapter is about that stretch of the journey. It covers three things: how often to sample, how to reduce noise without destroying the signal, and how to tell whether a reading is drifting rather than changing.

How Often to Sample

A sensor that is read every 10 ms is sampled at 100 Hz. The question is whether 100 Hz is enough, and the answer is given by the **Nyquist criterion**: to represent a signal faithfully, you must sample at more than twice the highest frequency present in it.

If a motor vibrates at 50 Hz and you sample at 60 Hz, you do not see 50 Hz. You see a slow wobble at 10 Hz, because 60 minus 50 is 10. This is **aliasing**, and it is pernicious because the false signal looks perfectly real. It is a smooth, plausible oscillation that is an artefact of the sampling, and no amount of filtering after sampling can remove it, because the information needed to distinguish it was never captured.

The defence has two parts. Sample faster than you think you need — a factor of five or ten above the bandwidth you care about is common. And filter the analogue signal before the converter, so that content above the sampling rate never reaches it.

Work an example. A vibration sensor is mounted on a gearbox that rings at 800 Hz. To see the ring in the data, the sample rate must exceed 1,600 Hz; 2 kHz is comfortable. A system that samples at 200 Hz will report a 0 Hz signal,

a flat line, while the gearbox rings. The data will look calm and the machine will be failing.

Noise, and the Square Root of N

Noise is the part of a measurement that changes when nothing in the world has changed. Some of it is thermal, some of it is electrical interference, some of it is quantisation, and all of it can be reduced — at a cost in time.

The simplest reduction is **averaging**. Take N readings and average them, and random noise falls by the square root of N. Sixteen readings reduce the noise by a factor of four, and sixty-four reduce it by a factor of eight. The cost is time: averaging sixteen readings at 1 kHz delays the answer by 16 ms, which is 24 mm of travel at 1.5 m/s.

A **moving average** does the same thing continuously, keeping the last N readings and reporting their mean at every step. It is easy to implement and it has a cost: it delays the signal by half its window, and it smooths genuine changes as well as noise.

A **low-pass filter** does the job more elegantly. A simple resistor-capacitor filter has a cut-off frequency of

```
cut-off = 1 / (2pi x R x C)
```

With a 1 kΩ resistor and a 10 µF capacitor, the cut-off is about 16 Hz: faster changes are attenuated, slower ones pass. In software, the same idea appears as an exponential filter that keeps a running estimate and moves it a fraction of the way towards each new reading. A factor of 0.1 gives a smooth signal and a lag of a few samples; a factor of 0.5 gives a fast signal and lets more noise through.

The choice of filter is always a trade between smoothness and delay, and the delay is the one that gets you into trouble. A robot that filters its distance readings heavily will detect an obstacle late, and no amount of accuracy makes up for reacting after the collision.

Where the Noise Comes From

Motors are the loudest thing in most robots, electrically speaking. Brushed motors produce sparks at the commutator, which radiate across a wide band. Switching drivers produce fast edges at tens of kilohertz. Long wires to a sensor act as antennas, and the shared ground connection carries every current back to the battery.

Four practical measures make an enormous difference and cost almost nothing.

Keep signal wires away from power wires. Route them on opposite sides of the machine, and cross at right angles when they must cross.

Twist signal pairs. A twisted pair picks up roughly the same interference on both wires, which cancels at the receiver.

Use a star ground. Every return path meets at one point near the battery, rather than daisy-chaining through the electronics. Ground loops — where two paths to ground exist — create currents that appear as noise in every signal that shares them.

Add decoupling capacitors near the electronics, so that a brief demand for current is supplied locally rather than through a long, inductive path.

None of these are advanced. All of them are missing from most student robots, and all of them show up as "the sensor is unreliable" when the sensor is fine.

Calibration: Zero, Span and Drift

A sensor reports a number; the robot needs a physical quantity. The mapping between them is established by **calibration**, and it has two parts.

Zero is the reading when the quantity is zero. A force sensor must read zero with nothing on it; if it reads 0.3 N, every measurement carries a 0.3 N offset.

Span is the change in reading per unit of the quantity. A temperature sensor that reports 10 mV per degree needs that figure, measured rather than assumed.

Calibration is usually a two-point procedure: measure at a known low value and a known high value, then fit a straight line. It sounds trivial until you discover that the sensor's response is not linear, that the span changes with temperature, and that the zero moves as the electronics warm up.

That last effect is called **drift**, and it is the reason serious instruments are calibrated before each use, or include a reference they can check against. A robot that measures force to decide whether it has gripped an object will fail if its force sensor has drifted by 2 N since the morning, because the threshold it compares against was set in the morning too.

The practical defence is a **zeroing routine**: with the gripper empty and still, take a hundred readings, average them, and subtract that value from every subsequent reading. It takes half a second, and it removes the largest single source of error in cheap sensors.

Problems

1. A signal contains content at 60 Hz. What is the minimum sampling rate needed to represent it? What happens if you sample at 100 Hz and what happens at 70 Hz?

2. You average 25 readings. By what factor does the random noise fall? If each reading takes 2 ms, what delay have you introduced?

3. Compute the cut-off frequency of a filter with a 4.7 kΩ resistor and a 1 µF capacitor.

4. A force sensor reads 0.4 N with nothing touching it. What would you do before using it, and what would happen if you did not?

5. Name three ways to reduce electrical noise in a robot without changing any software, and say which one is most likely to fix a sensor that misbehaves only when the motors run.

PART FOUR

Behaviour: From Commands to Conduct

CHAPTER TEN

Open Loop: Doing What You Are Told

The simplest robot does not check. It is told to turn the wheel for two seconds, and it turns the wheel for two seconds. Whether the robot has moved one metre or ninety centimetres, the program does not know and does not care.

This is **open-loop** control, and it is the right place to begin, for a reason that has nothing to do with simplicity. Open loop exposes the physics. When a machine driven by a fixed command consistently ends up 8 cm short, you have learned something concrete about friction, load or battery voltage. When the same machine is wrapped in feedback, that knowledge hides inside the correction, and the underlying problem stays in the machine, waiting to reappear when the battery is low.

The Command and the Consequence

Open-loop control has three parts: a trajectory of commands, a timing, and a hope.

For a wheeled robot, the command might be "left motor 60% forward, right motor 60% forward, for 2.0 s". If the wheel radius is 50 mm, one revolution covers 314 mm, and at 60% duty on a 12 V motor with a no-load speed of 300 revolutions per minute, the wheel turns at roughly 180 revolutions per minute, or 3 revolutions per second. Two seconds gives six revolutions, which is 1.88 m.

That number is a prediction, not a measurement. It assumes the motor's speed is proportional to duty cycle, that the floor is flat, that the wheels do not slip, that the battery is at 12 V, and that the robot's mass has not changed. Each assumption is slightly false, and the errors add.

Assumption	Typical error
Speed proportional to duty	5–10% over the range

Assumption	Typical error
No wheel slip	1–5% on a hard floor, far more on carpet
Battery at nominal voltage	10% drop from full to empty
Constant load	Varies with floor and payload

Multiply those errors and a robot commanded to travel 2 m may travel 1.75 m or 2.15 m. Neither is catastrophic, and both are enough to miss a docking station.

Dead Reckoning, and How It Drifts

A slightly better open-loop machine measures its own wheels with encoders and integrates. This is **dead reckoning**: if the wheels turned this much and did not slip, the robot moved this far.

For a two-wheeled robot, the arithmetic is small. Let the wheel radius be r, the distance between wheels b, and let each wheel's encoder count give its travel, s_left and s_right. Then

```
distance travelled = (s_left + s_right) / 2 change in heading = (s_right - s_left) / b
```

Take a robot with r = 0.05 m, b = 0.3 m, and 4,000 counts per revolution. The left wheel travels 1.000 m and the right 1.030 m over a straight-line command.

The centre moves (1.000 + 1.030)/2 = 1.015 m, and the heading changes by (1.030 - 1.000)/0.3 = 0.1 rad, or 5.7 degrees. The robot intended to go straight and has turned slightly, because one wheel ran faster.

Over a long run, this error compounds. A 2% difference between the wheels means that after 10 m the heading is off by 0.2 rad, about 11 degrees, and the robot is 1 m away from where it thinks it is. Add wheel slip, and the number gets worse quickly. This is why no mobile robot relies on dead reckoning alone, and why the volumes on localisation exist.

Profiles: Not All Commands Are Steps

Telling a motor to go from zero to full speed instantly is a bad idea, for the same reason that stamping on a car accelerator is. The mechanism must accelerate its own mass, the gears take a shock, and the structure rings.

The standard solution is a **motion profile**: a smooth change from one velocity to another. The simplest is the trapezoid — accelerate at a constant rate, cruise at a maximum speed, decelerate at the same rate.

Work an example for an arm joint that must move 1.0 rad. It accelerates at 2 rad/s², cruises at 1 rad/s, and decelerates at 2 rad/s².

The acceleration phase takes 0.5 s and covers 0.25 rad. The deceleration phase takes 0.5 s and covers 0.25 rad. The remaining 0.5 rad is covered at 1 rad/s, which takes 0.5 s. Total: 1.5 s, and the joint arrives with zero velocity and zero acceleration.

Without the profile — a step command — the joint would demand infinite acceleration, which the motor cannot supply, so it would saturate, lag, overshoot and ring. The profile does not make the machine faster. It makes the machine arrive where it was aimed.

Calibrating the Command

A command that is roughly right can be made accurate with a ruler and ten minutes, and the procedure is worth knowing because it converts an open-loop machine from a toy into an instrument.

Run the robot forward at 40% duty for exactly 2.0 s, ten times, and measure the distance travelled each time on a hard floor. Suppose the results are 0.94, 0.96, 0.93, 0.97, 0.95, 0.96, 0.94, 0.98, 0.95 and 0.94 m.

The mean is 0.952 m, and the spread from lowest to highest is 0.05 m, about 5%. Two numbers come out of this.

The **scale factor** is the true distance divided by the predicted distance. If the model predicted 1.00 m, the factor is 0.952, so the controller multiplies every commanded duration by 1.05 to correct it. The robot now travels where it is told on that floor, at that battery level, with that payload.

The **spread** is the honest limit of the method. A 5% variation means that repeating the same command puts the robot anywhere within a 5 cm band per metre. For docking to a charger that tolerance may be enough; for placing a part

it is not, and no calibration can fix it because the variation is not a constant offset. It is friction, carpet, battery sag and wheel slip changing from run to run.

Two refinements improve the result without adding a sensor. Compensate for battery voltage: measure the pack at the start of each run, and scale the command by the nominal voltage divided by the measured voltage. And calibrate at the temperature and load you will actually use, because a machine calibrated cold and light will overshoot when it is warm and loaded.

Trial	Distance (m)
1	0.94
2	0.96
3	0.93
4	0.97
5	0.95
6	0.96
7	0.94
8	0.98
9	0.95
10	0.94

The discipline here is the same one that appears in every volume of this series. Measure ten times, keep the mean and the spread, correct the systematic part, and state the random part honestly. A robot specification that says "accurate to 2%" without saying over what distance and on what floor is not a specification.

When Open Loop Is Enough

It would be a mistake to treat open-loop control as a beginner's technique to be outgrown. Several important machines use it deliberately.

A **stepper-driven axis** moves a fixed increment per pulse and is designed to hold that assumption. With a homing routine at start-up, it is accurate enough for 3D printers and pick-and-place machines, at a fraction of the cost of a servo axis.

A **fixtured industrial task** — welding, painting, screwdriving — happens in a world that has been built to be predictable. The part is clamped, the robot is bolted down, and the trajectory is repeated thousands of times. Feedback on the tool position would add cost without adding value, so the machine is open-loop in position and closed-loop only in the motor's own commutation.

A **conveyor or lift** moves between two limit switches, and the switches are the feedback. The motion between them is open loop, and it does not matter, because the endpoints are certain.

The engineering rule is: use open loop where the world is constrained and feedback where it is not. Then measure. A machine that is open loop and repeats to 0.1 mm is a better machine than one that is closed loop and argues with itself.

Problems

1. A robot with 50 mm wheels is commanded at 50% duty for 3 s. Assuming a no-load speed of 240 revolutions per minute at full duty and no slip, how far does it travel? What would you guess the real distance to be?

2. A two-wheeled robot has wheels 0.25 m apart and encoders giving 2,000 counts per revolution on 60 mm wheels. The left wheel records 1,200 counts and the right 1,260. Compute the distance travelled and the change in heading.

3. A robot's wheels differ in effective diameter by 1.5%. After travelling 20 m in a straight line, how far off course is it? Is that acceptable for a warehouse aisle 3 m wide?

4. Design a trapezoidal profile for a joint moving 2.0 rad, with a maximum speed of 1.5 rad/s and acceleration of 3 rad/s². How long does the move take?

5. Name one task in your own environment where open-loop control would be adequate, and one where it would fail badly. Explain the difference in terms of how predictable the world is.

CHAPTER ELEVEN

Closing the Loop: Simple Behaviours That Survive

A robot that checks its own result can survive a world it did not predict. The battery sags, the floor changes, a book is left on the carpet; the machine notices that it is not where it intended to be and corrects. This is **feedback**, and it is the difference between a machine that works in a laboratory and one that works on a Tuesday.

The mathematics of feedback fills several volumes. What follows is the smallest useful version: three behaviours you can build with one sensor, one motor and one line of arithmetic, and the reasons each of them behaves the way it does.

Error Is the Whole Idea

Every feedback controller does the same thing. It measures the difference between where it is and where it wants to be, and it acts to reduce that difference.

```
error = target - measured
```

If the target is 1.0 m from a wall and the sensor reports 0.85 m, the error is +0.15 m, and the robot should back away. If it reports 1.20 m, the error is -0.20 m, and it should approach.

The simplest controller acts on the sign of the error: move towards the target if the error is negative, away if positive. This is **on-off control**, and it is what a thermostat with a single threshold does. It works, and it chatters: the machine oscillates around the target because it has no sense of proportion.

Add proportion. If the error is large, act strongly; if it is small, act gently.

```
command = gain x error
```

This is **proportional control**, and the gain is the single number that decides the personality of the machine. Too low and it drifts towards the target, sluggish and always a little short. Too high and it overshoots, comes back, overshoots again, and eventually oscillates.

The Line Follower

A line follower is the classic first feedback robot, and it repays study because every part of it is visible.

Two light sensors sit either side of a black line on a pale floor. Each reports a value: high on pale floor, low on black line. The robot computes an error from the difference between them:

```
error = left - right
```

If both sensors see pale floor, the error is zero and the robot drives straight. If the left sensor sees the line, the robot has drifted right, so it steers left. The size of the steering command is proportional to the difference, so a small drift produces a small correction.

Now watch what happens as the gain rises. At a low gain, the robot follows gently and cuts corners. At a moderate gain, it tracks the line closely. At a high gain, it oscillates across the line, overcorrecting each time, and eventually leaves the track at a corner.

There is a second, subtler problem: a proportional controller always leaves a small error when a steady disturbance is present. On a slope, the robot needs a constant steering offset to hold the line, and a controller that only responds to error will settle slightly off-centre. The cure is an **integral** term, which accumulates the error over time and adds a correction until the error disappears. That term is the subject of the volume on control systems; here it is enough to know that it exists and that it is the reason a well-tuned machine does not sit with a permanent offset.

Adding the Integral Term, and What It Costs

The integral term accumulates error over time and adds a correction proportional to that accumulated total. Its purpose is to remove the steady offset that proportional control leaves behind.

Picture the line follower on a slightly sloped floor. To hold the line it needs a permanent steering offset, say 5% to the left. A proportional controller only produces output when there is an error, so it settles with a small permanent error: the robot tracks the line a few millimetres to one side forever. Add the integral term and the accumulated error grows until the offset is supplied, at which point the error returns to zero and the accumulation stops.

The cost is instability if the term is too strong. The integral term remembers the past, so it keeps pushing after the error has gone, overshoots, and then has to unwind. On a system with delay — and every robot has delay — a large integral gain produces slow, growing oscillations that are painful to diagnose because the proportional term looks correctly tuned.

Two practical rules keep it manageable. Clamp the accumulated value, so that a long period of saturated output does not build a huge total that takes seconds to discharge; this is called anti-windup and it is the difference between a robot that recovers from a jam and one that lurches when it is released. And add the integral term last, after the proportional gain is tuned on the flat floor, so that you can see what it changes.

Term	Responds to	Removes	Risk if too large
Proportional	Current error	Most of the error quickly	Oscillation
Integral	Accumulated error	Steady offset	Slow growing oscillation
Derivative	Rate of change of error	Overshoot	Amplifies noise

The derivative term, for completeness, acts on how fast the error is changing and damps the approach to the target. It is the term that makes a heavy machine stop without overshooting, and it is also the term that turns sensor noise into motor chatter, because differentiating a noisy signal amplifies the noise. Most small robots use proportional control alone, add integral when an offset appears, and add derivative only when the mechanics are stiff enough to tolerate it.

Reflexes and Battles

Some behaviours should not be polite. A bumper switch that reports contact should stop the robot immediately, whatever the navigation planner thinks.

This is a **reflex**: a fast, local response that overrides everything else. It is the robot equivalent of pulling your hand from a hot pan, and it should be implemented in the same spirit: short, simple, and impossible to talk out of.

The classic architecture for combining reflexes is **subsumption**, introduced by Rodney Brooks in the 1980s. Each behaviour runs continuously and proposes an action. Higher-priority behaviours suppress lower-priority ones. A robot might have a wander behaviour, a wall-following behaviour, and a halt-on-bump behaviour, in that order of priority. When the bumper closes, the halt wins, and it wins because of the wiring of the arbitration, not because the scheduler happened to run it.

Behaviour	Trigger	Action	Priority
Halt on bump	Bumper closed	Stop, reverse briefly	Highest
Avoid obstacle	Distance below 0.4 m	Turn away	High
Follow line	Line sensor active	Steer to line	Medium
Wander	Always	Drive forward slowly	Lowest

Subsumption fell out of fashion as planners became more capable, and it has never stopped being useful. Every safety layer in a modern robot is a subsumption system: a reflex that suppresses the intelligent behaviour when the reflex is triggered.

Timing: The Number Nobody Checks

A feedback loop has a period: the time between one measurement and the next. Everything about the behaviour depends on that period relative to how fast the machine can change.

A line follower cycling at 100 Hz sees the world every 10 ms. At 0.5 m/s it travels 5 mm per cycle, which is small compared with the sensor's field of view, and the control is smooth. The same robot cycling at 10 Hz travels 50 mm per cycle and will oscillate regardless of its gain, because it is always correcting a situation that has already changed.

The rule of thumb is that the loop should run at least ten times faster than the fastest thing the machine needs to respond to. For a line follower, 50 to 100 Hz is comfortable. For a balancing robot, 200 to 1,000 Hz. For a walking robot's joint, several kilohertz.

This is also the reason a control loop should never be slowed down by logging, printing or a network call. A `print` statement inside a loop that must run every millisecond is a bug, not a diagnostic.

Problems

1. A distance controller has a target of 1.0 m, a sensor reading of 0.9 m, and a gain of 0.5 per metre. What command does it produce? What command with the error reversed?

2. A line follower with a proportional gain of 10 oscillates. What are the two most likely causes, and what would you change first?

3. A balancing robot cycles at 100 Hz and falls over. Why is 100 Hz probably too slow, and what would you expect to happen at 500 Hz?

4. Sketch a subsumption architecture for a robot that must follow a corridor, avoid people, and stop when a button is pressed. List the behaviours in priority order and say what each one overrides.

5. Explain in two sentences why an integral term is needed on a slope but not on a flat floor.

CHAPTER TWELVE

When Robots Hurt People

On 25 January 1979, at a Ford casting plant in Flat Rock, Michigan, a worker named Robert Williams climbed into a storage rack to retrieve parts. A robot arm on the other side of the rack continued its programmed motion and struck him. He died later that day. It was the first recorded death caused by an industrial robot in the United States, and the details are more instructive than the headline: the machine was doing exactly what it had been built to do, in a place where a person had no way to tell it that someone was there.

Two years later, in Akashi, Japan, a maintenance worker named Kenji Urada was killed at a Kawasaki Heavy Industries plant. He had climbed over a fence to work on a robot without switching it off completely. The machine's hydraulic arm pushed him into an adjacent grinding machine.

Neither accident involved a malfunctioning robot. Both involved a person inside the working envelope of a machine that had no way to know, and a set of procedures that allowed the situation to arise. That pattern has not changed in forty-five years, and it is the reason this chapter is not an appendix.

How Machines Fail

Before designing a guard, it helps to know what is being guarded against. Robot failures fall into five groups, and the groups have different characters.

Mechanical failures are broken parts: a fatigued bracket, a sheared key, a bearing that seizes, a bolt that vibrates loose. They are usually preceded by warning signs — noise, heat, vibration — and they are usually found by inspection rather than by software.

Electrical failures are shorts, opens, insulation breakdowns and connector faults. Vibration is the great enemy of connectors, and the machine that works on the bench and fails on the floor often has a connector that half-disconnects when the wheel hits a bump.

Software failures are logic errors, race conditions, unbounded loops and unhandled states. They are the most discussed and the least common cause of injury, because most injuries require a physical energy source and the software is usually not one.

Sensing failures are the dangerous middle ground. A sensor that reports stale data, an encoder that misses counts, a camera blinded by the sun. The machine believes something false and acts confidently on it.

Human failures are the largest category in practice: someone reaches past a guard, forgets to lock out power, presses the wrong button, or trusts the machine's confidence more than their own eyes.

The engineering response differs by category. Mechanical and electrical failures are prevented by design margins and inspection. Sensing failures are prevented by redundancy and plausibility checks. Human failures are prevented by design as well, because the best way to stop someone defeating a guard is to make the guard unnecessary.

Risk, in Two Numbers

A **risk assessment** is a structured way of asking what could go wrong, how bad it would be, and how likely it is. The simplest useful version scores each hazard on two scales and multiplies them.

Hazard	Severity (1–5)	Likelihood (1–5)	Risk score
Arm strikes a person during normal cycle	5	2	10
Pinch point at the gripper	3	4	12
Hot motor surface touched during maintenance	2	3	6
Robot drives off a mezzanine edge	5	1	5
Cable trip hazard around the cell	2	5	10

The scores are not physics; they are a disciplined conversation. What they buy is a ranking, which tells you where to spend the effort, and a record, which tells a regulator, an insurer and a court that the conversation happened.

Two things are worth noticing about the table. First, the highest score is not the most dramatic hazard: the pinch point at the gripper outranks the arm strike

because it is far more likely. Second, the edge hazard has the highest severity and the lowest likelihood, which is characteristic of the failures that kill people: rare, and fatal.

The Order of Controls

Engineers have a hierarchy of ways to make a machine safe, and the order matters. Each step is more reliable than the one below it.

Eliminate the hazard. Design the machine so the dangerous thing cannot happen. A robot that carries the part on a conveyor instead of reaching into a press has removed a hazard rather than guarded it.

Guard it. Put a physical barrier between the person and the energy. A fence around a welding cell is crude, cheap and extremely effective.

Interlock it. Make the machine stop or slow when a person is detected. A light curtain, a pressure mat, a scanner. An interlock is a guard with a sensor, and it is only as good as the sensor.

Warn. Lights, sirens, floor markings, signs. Warnings are the weakest control, because they depend on attention, and attention is exactly what a tired worker at the end of a shift has least of.

Train and instruct. Procedures, permits, lockout-tagout. Necessary, and insufficient alone, as both the 1979 and 1981 deaths demonstrate.

Personal protective equipment. The last line, and the least reliable, because it protects only the person wearing it and only if it fits.

A design that relies on the bottom three while calling itself safe is a design that has moved the risk onto the person least able to bear it.

What a Real Safety System Contains

A modern industrial robot cell has several independent layers, and each one is designed to work when the others fail.

The **emergency stop** cuts power to the actuators through a contactor, not through software. It is latching, so that restarting is deliberate, and it is placed where a person would naturally reach.

The **enabling device** is the dead-man switch held by a programmer inside the cell. Release it, and the robot stops. It is how a person is allowed to be near a moving machine at all.

The **light curtain** or **scanner** detects an intrusion and stops or slows the machine before contact. Its resolution determines the smallest object it can detect, and therefore the minimum safety distance.

The **speed and separation monitor** is the modern refinement used in collaborative cells: instead of stopping, the robot slows as a person approaches, keeping a calculated minimum distance. It requires reliable human detection, and its safety depends entirely on the perception system.

The **mechanical brake** holds the arm when power is removed, so that gravity does not finish what the fault started.

The **force and torque limit** in a collaborative robot stops the motion when contact produces a force above a threshold. It is the last layer, and it is designed to make the residual injury minor rather than fatal.

The relevant standards are worth knowing by name even if you never read them in full. ISO 10218 covers industrial robots and their integration. ISO/TS 15066 covers collaborative operation and sets out force and pressure limits for contact with different parts of the body. ISO 13849 defines performance levels for safety-related control systems, from PLa to PLe. IEC 61508 defines safety integrity levels for electrical and electronic systems. These are not formalities; they encode decades of accidents.

Human Factors

The hardest safety problem is not the machine. It is the person who has been asked to watch it.

Vigilance does not survive boredom. A supervisor asked to watch an autonomous vehicle for hours will look away, and the more reliable the vehicle, the more often they will look away. This is not a moral failing; it is a well-documented property of human attention, and it is why the Tempe crash in 2018 happened with a safety driver who was not looking at the road.

Automation bias is the tendency to trust the machine's judgement over your own, especially when it is usually right. A surgeon who has seen the robot be correct a thousand times will hesitate before overriding it on the thousand-and-first, and the hesitation is the accident.

Mode confusion is the third. A machine that behaves differently in automatic and manual modes, with no clear indication of which mode it is in, will eventually be operated in the wrong one. The fix is design: unambiguous state indic-

ation, and controls that are physically different for different modes.

None of these are solved by better software. They are solved by designing the human's job so that it is possible to do it well.

Problems

1. Take the risk table above and add three hazards from a robot you have seen or used. Score them and re-order the list.

2. A light curtain has a response time of 20 ms, and the robot takes 200 ms to stop from full speed. If the robot moves at 1.5 m/s, what is the minimum safety distance?

3. Explain in three sentences why an emergency stop should cut power through a contactor rather than ask the software to stop.

4. A collaborative robot is rated for a maximum contact force of 140 N. What could make that rating meaningless in a real installation, and what would you check?

5. Describe one task in which a human supervisor is asked to monitor a machine for long periods. What would you change about the task to make vigilance realistic?

PART FIVE

Making and Using

CHAPTER THIRTEEN

Building Your First Robot

There is a moment, about three weeks into a first robot, when the parts are all on the bench and nothing works. The motors spin when tested alone; the sensors read sensibly when tested alone; the code runs on the desk. Assembled, the robot twitches, resets, or does nothing at all.

That moment is not a sign of failure. It is the actual work of integration, and the way through it is method rather than inspiration. This chapter is a build procedure: what to write down before you buy anything, in what order to assemble, and how to test so that when something breaks you know which part broke.

Start With a Brief, Not a Kit

A robot begins as a sentence: *this machine will carry a 2 kg box across a flat floor for 90 minutes, at walking pace, and stop within 10 cm of a marker.*

That sentence contains a task, a payload, a duration, a speed and an accuracy. Each of those numbers will drive a decision later, and a project without them will drift into whatever the parts allow. Write the brief before the shopping list, and keep it where you can see it.

Requirement	Target	Why it matters
Payload	2 kg	Sets motor torque and structure
Speed	1.0 m/s	Sets gearing and control loop rate
Runtime	90 min	Sets battery capacity and mass
Stopping accuracy	0.1 m	Sets sensor choice and control method
Floor	Flat, indoor	Rules out suspension and rough-terrain design
Budget	Fixed	Forces the trade-offs early
Safety	Stops on contact	Sets bumper placement and firmware

Two of these numbers, speed and runtime, will fight each other through the battery and the motors. A faster robot covers more ground per second and therefore needs more power; a longer runtime needs a bigger battery, which adds mass, which needs more torque, which draws more current, which shortens the runtime. Every mobile robot design is a resolution of that loop, and the resolution is arithmetic, not opinion.

Choosing the Shape

Three body plans cover most first projects.

A **differential drive** robot has two powered wheels and one or more castors. It can turn in place, it is simple to control, and it is hopeless on rough ground. It is the right choice for a first mobile robot, because the mathematics of its motion is two lines long.

A **four-wheeled** robot is more stable and harder to turn, because the wheels fight each other unless one axle can slip or steer. It is the right choice when stability matters more than manoeuvrability.

An **arm on a base** replaces wheels with joints. It does not go anywhere, and it can do far more interesting things with objects. It is the right choice when the task is manipulation rather than transport.

Whichever you choose, decide early where the heavy parts go. The battery and the electronics belong low and near the centre, because mass high up makes the machine tippy and mass far from the drive wheels reduces grip.

The Bill of Materials

A bill of materials is a table, and it is the most useful document in the project. Here is an illustrative one for the brief above, with prices that are typical rather than exact.

Item	Quantity	Unit cost	Notes
Geared motor, 12 V, 200 rpm	2	18	With encoder if possible
Wheel, 100 mm	2	6	Matches the gearing calculation
Castor wheel	2	3	Low profile
Motor driver, dual H-bridge	1	15	Must handle stall current

Item	Quantity	Unit cost	Notes
Microcontroller board	1	12	Enough pins and a hardware timer
Battery, 11.1 V, 5 Ah	1	40	With management board and fuse
Chassis plate	1	20	Aluminium or plywood
Bumper switches	2	3	Mechanical stop detection
Distance sensor	3	12	Front and sides
Fasteners, wire, connectors	—	25	Budget more than you think
Switch, fuse holder, emergency stop	1 set	20	Do not omit these

The total is around 220 in the currency of your choice, which is roughly the price of a decent bicycle and about a tenth of a commercial research platform. The proportions matter more than the total: the battery, the motors and the driver are most of the cost, and they are the parts that decide whether the machine works.

Assembly in Order

Build in the order that lets each stage be tested before the next is added. This is the single most important habit in the chapter.

Stage 1: the frame. Assemble the chassis, wheels and castors with no electronics. Push it. Does it roll straight? Does it wobble? Fix it now, when the fix is a drill and not a redesign.

Stage 2: the motors, on the bench. Connect one motor to the driver and the driver to a power supply with a current limit. Run it forward, backward, slow, fast. Measure the current at stall with the shaft held. Compare with the datasheet.

Stage 3: the electronics, unpowered by the motors. Power the controller alone. Read one sensor. Print its values. Then read all sensors and check for plausible numbers, not just numbers.

Stage 4: the drivetrain under load. Put the robot on blocks so the wheels spin free, then on the floor. Command a slow forward motion for one second. Measure how far it actually went, and compare with the prediction from the chapter on open loop.

Stage 5: behaviour. Add the first feedback loop. Tune the gain on a smooth floor before trying carpet.

Stage 6: the safety layer. Add the bumpers, the stop button and the current limits. Test them by deliberately triggering each one, and confirm that the robot stops.

Each stage ends with a test whose failure is informative. A robot assembled in one go and switched on has a single symptom — "it does not work" — and no way to narrow it down.

When It Misbehaves

Integration faults have a small number of usual suspects, and the order of checking saves hours.

Power. Measure the battery voltage under load, not at rest. A pack that reads 12.4 V with no load may read 10.1 V when the motors start, which is below the controller's minimum.

Grounds. Check that every component shares a single ground point. A missing ground produces signals that wander; two grounds produce loops that hum.

Connectors. Wiggle every connector while the robot is running. Intermittent faults live here, especially after the robot has driven over a bump.

Noise. Does the fault appear only when the motors run? If so, it is electrical noise, and the fixes are physical: separate the wires, add capacitance, twist the signal pair.

Logic. Only after the four above should you suspect the code. Software is easier to inspect than a loose crimp, which is precisely why people inspect it first and find nothing.

Document While You Build

A robot without documentation is a robot that cannot be repaired, and a project report without measurements is an essay. Keep three things as you go.

A **log**, one line per test: date, what you changed, what happened, what you measured. It takes a minute and saves a week.

A **wiring diagram**, drawn by hand and kept next to the machine. The version in your head will not survive a week.

A **bill of materials with the actual parts**, including the ones you substituted, because the substitute is what is in the machine.

Add version control for the code, even if you are working alone. The ability to return to the version that worked yesterday is worth more than any clever feature you might add today.

Problems

1. Write a one-sentence brief for a robot you would like to build, containing a task, a payload, a speed, a duration and an accuracy.

2. Derive the motor torque needed for your brief. Show the mass, the distance and the acceleration you assumed.

3. Estimate the battery capacity your brief requires, using the runtime and an average current you justify.

4. List the six assembly stages for your robot and the test that ends each one.

5. Your robot resets when it drives forward. Write the four checks you would perform, in order, and what result would confirm each cause.

CHAPTER FOURTEEN

Robots at Work and at Home

The word "robot" carries a science-fiction shadow that makes the real machines look disappointing. The real machines are more interesting, because they had to solve the problems of dirt, heat, vibration, cost and insurance, and the solutions are visible in their shapes.

Look at the machines that already work for a living and you will see the same three questions answered again and again: what is the energy source, what is the world like, and who is responsible when it goes wrong.

The Arm Behind the Fence

The industrial arm is the most successful robot ever built by the numbers: several hundred thousand are in service, and the number grows every year. The typical machine lifts 6 to 200 kg, reaches 1 to 3 m, and repeats a position to between 0.02 and 0.1 mm, tens of thousands of times without rest.

Its design follows from a single decision: the arm is bolted to the floor and the work comes to it. That removes localisation, perception, battery and navigation from the problem, and it leaves speed, accuracy and reliability — the three things industry will pay for.

The economics are simple enough to compute. A welding cell might cost 150,000 including the robot, the fixture, the fence and the integration. It runs two shifts, five days a week, which is roughly 3,600 hours a year. If it replaces two operators whose fully loaded cost is 30,000 each, the payback is a little over two years, and the machine then runs for a decade with maintenance. This arithmetic, rather than any interest in intelligence, explains why the arm exists.

Its weakness is equally simple. The arm knows nothing about parts that are not where the fixture says they are. A part presented at a random orientation defeats it completely, which is why the field has spent thirty years trying to add eyes, and why the volumes on vision and manipulation exist.

The Warehouse

In 2012, Amazon bought a company called Kiva Systems for about 775 million dollars. Kiva made small, flat, orange robots that carried shelving units across a warehouse floor. Instead of a robot reaching a shelf, the shelf came to the picker.

The engineering is instructive because it is so constrained. The robots follow barcode-like markers on the floor, so their localisation is trivial. They move slowly and carry a lot, so their motors are modest. They cooperate by not colliding, using traffic rules and a central coordinator rather than sophisticated negotiation. The result is a system that increased picking productivity severalfold, and it works because the warehouse was rebuilt around the robots rather than the robots being dropped into an existing warehouse.

That last point is the one most often missed. The successful deployment changed the building, the shelving, the workflow and the job of the people in it. A robot that has to fit into an unchanged world is a much harder problem, and often a worse investment.

Where Precision Is the Point

Surgical robots, led by the da Vinci system approved in 2000, do not operate autonomously. A surgeon sits at a console and moves instruments through a robot, which scales motion down, removes tremor and allows access through small incisions.

The robot's sensors report force and position; its actuators are precise and smooth; its failure modes are designed around the surgeon's control. The safety question is different from an industrial cell, because the patient is inside the working envelope. The answer is not a fence but a set of limits on force, speed and range, plus a control architecture that always leaves the surgeon in command.

Surgical robotics is the clearest case in the field for teleoperation: the value is not autonomy but precision, dexterity and the ability to see. Attempts to automate any part of the procedure run into the same wall as everywhere else, which is that the world inside a body is not the world of the training data.

Outdoors

Agricultural robots, drones and mining machines work in places that are deliberately unstructured. They succeed by narrowing the task.

A field robot that removes weeds recognises one plant and ignores the rest, and it does so with a camera, a light source and a sprayer, at a speed slow enough to allow the perception to work. A drone that maps a field flies a planned grid and takes photographs; the intelligence is in the stitching on the ground. A mining truck follows a route that has been driven a thousand times and uses radar to avoid the other trucks.

The pattern is consistent: outdoors, the world is unpredictable, so the successful machines reduce the number of decisions they make and take their time over the ones that remain.

At Home

A domestic robot vacuum sells in the tens of millions, and it is the closest thing the field has to a mass-market mobile robot. It also illustrates every constraint in this volume at once.

It must cost a few hundred, so its motors are small, its sensors are cheap and its structure is plastic. It must run on a battery, so its power budget is tight and its suction is modest. It must operate around furniture, pets and cables, so it must fail gracefully: bumping into things, getting stuck, and asking for help rather than damaging the house.

The generations of improvement follow the ladder from Volume One. Early models bounced at random. Later models mapped, planned coverage, and returned to a dock. Each step required better sensing, more computation and more careful power management, which is to say it required everything this volume has described.

The Human Beside the Machine

The fastest-growing category of industrial robot is the one designed to be next to a person rather than behind a fence. A collaborative arm weighs twenty to thirty kilograms, lifts five to fifteen, and is limited in speed and force so that a contact with a human is uncomfortable rather than injurious.

The economics changed when these machines arrived. A traditional cell needs a fence, a safety relay, a fixture and an integrator, and the installation often costs more than the robot. A collaborative arm can be wheeled to a bench, taught by guiding it through the motion by hand, and put to work the same day. For a small factory making a hundred different products in small batches, that difference is decisive.

The safety argument is subtler than the marketing suggests. A collaborative rating describes what happens when the robot contacts a person under specified conditions: the force and pressure must stay below the limits in the standard, and the limits depend on which part of the body is contacted. A rounded forearm touching an arm is one case; a sharp gripper edge touching a face is another. The rating belongs to the whole application, not to the robot alone, and the tool, the workpiece and the fixture can each destroy it.

The human factors are the other half. A person working beside a machine that moves unpredictably will be startled, and a startled person makes mistakes. Good installations make the robot's intentions visible: it slows before it turns, it pauses when a hand enters its path, it signals with light or sound before a fast move. The machine does not need to be charming. It needs to be legible.

Why Machines Still Fail at Work

Three limits recur across every application.

The world is not the training ground. Every robot meets conditions its designers did not anticipate, whether that is a new part, a wet floor, a different light or a person behaving unusually.

Integration is the cost. The robot is often a small fraction of the project. Fixtures, safety systems, software, training and maintenance are the rest, and they are where budgets overrun.

Responsibility is unresolved. When a machine injures someone, the question of who is liable — manufacturer, integrator, owner, operator — is answered by law and contract rather than by engineering, and the answer shapes what gets built. A machine whose failure has no clear owner tends not to be deployed.

Problems

1. An industrial cell costs 150,000 and replaces two operators at 30,000 each per year. Compute the payback period, assuming 10% annual maintenance. What would make the investment fail?

2. A warehouse robot moves at 1.2 m/s and carries 500 kg. Estimate the kinetic energy at full speed and explain why the safety system must be designed around it.

3. A domestic vacuum robot has a 14.4 V, 2.5 Ah battery and draws 25 W on average. Estimate its runtime, and say what would change if it drew 40 W.

4. Name one task in your country that robots do not currently do and explain, in three sentences, whether the obstacle is technical, economic or social.

5. For the surgical robot described above, list three failure modes and, for each, the layer of protection you would expect to find.

Glossary

Absolute encoder — a sensor that reports the full joint angle at every reading, without needing a reference position.

Actuator — the part of a machine that changes the world; in this volume, usually a motor.

Backlash — the small free movement between gear teeth before the output begins to turn.

Ball screw — a screw transmission using recirculating balls, with high efficiency and low backlash.

Bill of materials — the list of every part in a build, with quantities and costs.

Brushless motor — a motor with electronic commutation instead of brushes; efficient, durable and dependent on a driver.

Cantilever — a beam fixed at one end and free at the other; the basic model of a robot link.

C-rate — the multiple of a battery's capacity at which it can be discharged.

Dead reckoning — estimating position by integrating measured motion, without an external reference.

Degrees of freedom — the number of independent motions a mechanism can make.

Differential drive — a mobile robot driven by two independently powered wheels.

Duty cycle — the fraction of time a machine or signal is active; for motors, the fraction of time a load may be applied.

Encoder — a sensor that reports rotation as a series of counts.

End effector — the tool at the end of a robot arm: a gripper, a welder, a camera.

Frame — a coordinate system attached to a part of the machine.

Gear ratio — the ratio of input turns to output turns in a transmission.

H-bridge — four switches arranged to drive a motor in both directions.

Harmonic drive — a compact, high-ratio, near-zero-backlash transmission used in robot joints.

Inertial measurement unit — a sensor package reporting acceleration and angular rate.

Joint — a connection between links that permits some relative motion.

Kinematic chain — the sequence of links and joints from base to tool.

Lead screw — a screw that converts rotation into linear motion at a fixed advance per turn.

Nyquist criterion — the rule that a signal must be sampled at more than twice its highest frequency.

Open-loop control — control without feedback: the command is issued and not checked.

Payload — the mass a robot can carry at its tool, usually specified at a given reach.

PWM — pulse width modulation; rapid switching used to control average voltage.

Parallel mechanism — a mechanism connecting base and platform through several chains at once.

Prismatic joint — a joint that allows sliding along one axis.

Proportional control — a controller whose output is proportional to the error.

Repeatability — the ability to return to the same point; distinct from accuracy.

Resolution — the smallest change an instrument or actuator can report or produce.

Revolute joint — a joint that allows rotation about one axis.

Risk assessment — a structured evaluation of hazards by severity and likelihood.

Serial mechanism — a mechanism whose links form a single chain from base to tool.

Servo — a motor with position feedback and a controller that holds a commanded position.

Shunt resistor — a low-value resistor used to measure current.

Stepper motor — a motor that moves in fixed increments and holds position without feedback.

Stiffness — resistance to deflection under load; distinct from strength.

Subsumption — an architecture in which higher-priority behaviours suppress lower-priority ones.

Teleoperation — direct human control of a remote or hazardous machine.

Torque — a turning force, measured in newton-metres.

Workspace — the set of positions a robot's tool can reach.

Watt-hour — a unit of energy: one watt supplied for one hour.

Sources and Further Reading

The factual claims in this volume are drawn from the works below, and from the standards named in the chapter on safety. Component figures are typical values from manufacturers' datasheets; where a number is an illustrative example, the text says so.

Books on robot mechanics

Craig, J. *Introduction to Robotics: Mechanics and Control*. The standard treatment of frames, kinematics and Jacobians; Chapter 1 to 3 pair well with this volume.

Siciliano, B., and Khatib, O., eds. *Springer Handbook of Robotics*. Comprehensive, expensive, and the right place to look up a mechanism you have not met before.

Norton, R. *Design of Machinery*. For linkages, cams and four-bars, with the degree-of-freedom counting done properly.

Shigley, J., and Mischke, C. *Mechanical Engineering Design*. The source for deflection, fatigue and shaft design when a robot link must survive ten million cycles.

Motors, drives and power

Hughes, A., and Drury, B. *Electric Motors and Drives*. Clear on the torque-speed relationship and on what a motor's ratings actually mean.

Krishnan, R. *Electric Motor Drives: Modeling, Analysis and Control*. For readers who want the electronics and the control together.

Linden, D., and Reddy, T., eds. *Handbook of Batteries*. The reference on chemistry, capacity and safety.

Electronics and embedded systems

Horowitz, P., and Hill, W. *The Art of Electronics*. The book people keep on the bench, for the chapters on noise, grounding and power supplies.

White, E. *Making Embedded Systems*. Practical guidance on interrupts, timing and the discipline of small systems.

Safety and standards

ISO 10218-1 and ISO 10218-2. *Robots and robotic devices — Safety requirements for industrial robots.*

ISO/TS 15066. *Robots and robotic devices — Collaborative robots.*

ISO 13849-1. *Safety of machinery — Safety-related parts of control systems.*

IEC 61508. *Functional safety of electrical/electronic/programmable electronic safety-related systems.*

Occupational Safety and Health Administration. Investigations and publications on robot-related fatalities, including the 1979 Ford case.

Robotics in practice

Brooks, R. *Cambrian Intelligence*. The collected papers on subsumption, still the clearest statement of behaviour-based robotics.

Thrun, S., Burgard, W., and Fox, D. *Probabilistic Robotics*. For the localisation and mapping that this volume deliberately leaves out.

Wurman, P., D'Andrea, R., and Mountz, M. (2008). “Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses.” *AI Magazine*. The Kiva system described by the people who built it.

Every calculation in this volume can be reproduced with a calculator. Where a component's rating is quoted, it is a typical figure for that class of part rather than a specification of any particular product.